# Model Context Protocol

## Complete Interview Preparation Notes

Concepts · Architecture · Tested Code · Interview Q&A

Original reference notes merged with the guided code-first lessons

Edition: October 2026

# Table of Contents

**About This Edition . . . 6**

How the guided lessons were merged . . . 6

What changed compared with the first version of these notes . . . 6

How to read the boxes . . . 7

The learning method used throughout . . . 7

**1. Introduction to MCP . . . 8**

1.1 What is MCP? . . . 8

1.2 Why MCP was created . . . 8

1.3 The integration problem MCP solves (the N x M problem) . . . 8

1.4 MCP is a protocol, not an AI framework . . . 9

1.5 Why does the LLM need MCP at all? . . . 9

1.6 An MCP server is not "a REST API", and not an LLM . . . 10

1.7 Why MCP when LangGraph already has tools? . . . 10

1.8 Write once, reuse across AI applications (with code) . . . 12

1.9 MCP vs REST: the "AI-facing contract" . . . 14

1.10 MCP vs traditional approaches . . . 15

1.11 Key benefits . . . 15

1.12 The three primitives to remember first . . . 15

**2. MCP Architecture . . . 15**

2.1 The three roles: Host, Client, Server . . . 16

2.2 Host in depth . . . 16

2.3 MCP Client in depth . . . 16

2.4 MCP Server in depth . . . 17

2.5 One host, many clients . . . 17

2.6 Communication flow (step by step) . . . 17

2.7 One complete request, traced . . . 18

2.8 Terminology traps (do not say these in an interview) . . . 18

2.9 JSON-RPC 2.0 at a glance . . . 18

2.10 Host, Client and Server compared with LangGraph . . . 19

2.11 Interview questions on the architecture . . . 19

**3. Transport Layer . . . 19**

3.1 Protocol vs transport: the key separation . . . 20

3.2 Why we need more than one transport . . . 20

3.3 stdio transport . . . 21

3.4 stdio in code (tested) . . . 22
3.5 HTTP transport and Streamable HTTP . . . 24
3.6 HTTP+SSE (legacy) . . . 26
3.7 Choosing a transport . . . 26
3.8 Internals: how the two paths differ . . . 26
3.9 The transport knows nothing about your business system . . . 27
3.10 Interview questions on transports . . . 27
3.11 Exercises . . . 28

**4. Core Primitives . . . 28**
4.1 The six primitives at a glance . . . 28
4.2 Resources . . . 29
4.3 Tools . . . 34
4.4 Prompts . . . 35
4.5 Sampling . . . 40
4.6 Roots . . . 41
4.7 Elicitation . . . 41
4.8 Choosing between Tool, Resource and Prompt . . . 41

**5. Building an MCP Server . . . 41**
5.1 Setup . . . 41
5.2 The minimal server . . . 42
5.3 Design the tools before writing code . . . 42
5.4 The full server (tested) . . . 42
5.5 Inspect and debug with MCP Inspector . . . 44
5.6 A second example: a weather server . . . 44
5.7 The low-level Server API . . . 45
5.8 Trace one tool call through the server . . . 45
5.9 Mapping the pattern to enterprise systems . . . 46
5.10 Does the client execute the Python function? . . . 46
5.11 Production design: what must change before deployment . . . 46
5.12 Interview questions on building servers . . . 47

**6. Building an MCP Client . . . 48**
6.1 What a client does . . . 48
6.2 The full client for the claims server (tested) . . . 48
6.3 Capability discovery pattern . . . 50
6.4 Reading results correctly . . . 50
6.5 Direct call vs session.call_tool . . . 51
6.6 Practice tasks . . . 51

**7. Protocol Messages Deep-Dive . . . 51**

7.1 The problem: how does a client tell a server to run a function? . . . 51
7.2 What is JSON-RPC 2.0 . . . 51
7.3 The three message types . . . 52
7.4 A tiny JSON-RPC dispatcher in Python (tested) . . . 53
7.5 What the SDK does when you call session.call_tool . . . 56
7.6 The tool lifecycle: tools/list, then tools/call . . . 57
7.7 Does MCP replace REST? . . . 59
7.8 Message reference tables . . . 59
7.9 Interview questions on the protocol . . . 60
7.10 Practice . . . 61

**8. Authentication and Security . . . 61**

8.1 Authentication vs authorisation . . . 61
8.2 Tool discovery does not grant permission . . . 61
8.3 OAuth 2.1 in MCP (HTTP transports) . . . 61
8.4 A real token-protected MCP server (tested) . . . 62
8.5 HTTP vs stdio security . . . 66
8.6 Security best practices . . . 66
8.7 What the guided Lesson 8 audit found (and how it is now closed) . . . 66
8.8 Lesson 8 checklist . . . 67
8.9 Interview questions on security . . . 67

**9. Advanced Features . . . 68**

9.1 Pagination . . . 68
9.2 Progress reporting and logging (tested) . . . 68
9.3 Cancellation . . . 69
9.4 Change notifications and subscriptions . . . 69
9.5 Batch requests . . . 69
9.6 Sampling, Roots and Elicitation in code . . . 70

**10. Error Handling, Timeouts, Retries and Reliability . . . 70**

10.1 The four things to handle . . . 70
10.2 The failure flow . . . 71
10.3 Retries with exponential backoff and jitter (tested) . . . 71
10.4 Timeouts (tested) . . . 73
10.5 Applying both to a real MCP client call (tested) . . . 73
10.6 Circuit breaker (tested) . . . 75
10.7 Making the whole workflow reliable . . . 76

10.8 Where does each failure occur? (layers) . . . 77
10.9 Interview questions on reliability . . . 77
10.10 Lesson 9 checklist . . . 78

**11. MCP with LangChain and LangGraph . . . 79**
11.1 MCP vs LangChain vs LangGraph . . . 79
11.2 Architecture: follow a request . . . 79
11.3 Build it . . . 79
11.4 Internals: how an MCP tool becomes a LangGraph tool . . . 84
11.5 A second AI application reusing the same server . . . 84
11.6 Production considerations . . . 85
11.7 Interview questions . . . 85
11.8 Lesson 10 checklist . . . 86

**12. Tools, Resources and Prompts in One AI Workflow . . . 87**
12.1 The three primitives, again . . . 87
12.2 Architecture . . . 87
12.3 The server: all three primitives (tested) . . . 87
12.4 The host application (tested with --dry-run) . . . 89
12.5 Modify, break and fix . . . 91
12.6 Internals: what actually crosses the MCP boundary . . . 92
12.7 Production considerations . . . 92
12.8 Interview questions . . . 92
12.9 Lesson 11 checklist . . . 93

**13. Popular MCP Servers . . . 93**
13.1 Configuring servers in Claude Desktop . . . 94

**14. Interview Questions and Answers (Consolidated) . . . 95**
14.1 The 30-second answers . . . 95
14.2 Cross-cutting questions . . . 96
14.3 Rapid-fire definitions . . . 97
14.4 Quick reference cheat sheet . . . 98

**Appendix A: Study Roadmap and Progress . . . 99**

**Appendix B: Findings From Running the Code . . . 100**

**Appendix C: Code Files and How to Run Them . . . 101**

# About This Edition

This PDF merges two things into one document: the original **MCP reference notes** and the **guided, code-first lessons** (Lessons 1 to 11) you worked through in the study chat. Where a guided lesson belonged inside an existing section it was merged there; otherwise it became a new section. Nothing from the lesson transcript was dropped: every concept, diagram, table, code sample and interview question is here, reorganised so that each topic lives in one place.

## How the guided lessons were merged

| Guided lesson (study chat) | Where it lives in this document |
|---|---|
| Lesson 1: what MCP is, plus the follow-up doubts (why MCP when LangGraph has tools, reuse across apps, REST vs MCP) | Section 1 |
| Lesson 2: Host, Client, Server | Section 2 |
| Lesson 3: protocol and JSON-RPC (concept and code) | Section 2.9 (overview) and Section 7 (in depth, with code) |
| Lesson 4: transports (concept and code) | Section 3 |
| Lesson 5: build a server and client; tools/list and tools/call lifecycle | Sections 5, 6 and 7.6 |
| Lesson 6: Resources | Section 4.2 |
| Lesson 7: Prompts | Section 4.4 |
| Lesson 8: security and authorization | Section 8 |
| Lesson 9: errors, timeouts, retries, reliability | Section 10 (new) |
| Lesson 10: MCP with LangChain and LangGraph | Section 11 (new) |
| Lesson 11: Tools, Resources and Prompts in one workflow | Section 12 (new) |

## What changed compared with the first version of these notes

- Every runnable code sample from the guided lessons was **actually executed** (Python 3.13, `mcp` 1.30.0, `langchain-mcp-adapters` 0.3.2). The outputs shown under the **OUTPUT** boxes are real captured output, not predictions. The only samples not executed are the ones that need something unavailable here (an Azure OpenAI key, internet access, or a host that implements sampling and elicitation); each of those is labelled "not executed" where it appears.
- The lesson transcript was cut off in several places (the pasted code stopped mid-file). Those snippets were completed and tested.
- Running the code surfaced behaviours the transcript got slightly wrong or did not mention. These are collected in **Appendix B** and flagged inline with a **WATCH OUT** or **TESTED** box. The important ones: `resources/list` does not return URI templates; `isError` is `false` for business errors you return as data; a plain `-> dict` tool returns JSON as text, not structured content.
- The first version used a font without arrows and symbols, so some characters printed as black boxes. This edition uses full Unicode fonts, real indentation in code, and a clickable table of contents with page numbers.

## How to read the boxes

**TIP** A practical shortcut or good habit. NOTE: Background that helps you understand the surrounding text. KEY: The one idea to remember from the section. INTERVIEW: How to say it in an interview. TESTED: Code or behaviour verified by running it. VERSION: Something that depends on the SDK or protocol revision. WARN: A common mistake or a production risk.

## The learning method used throughout

Every guided lesson follows the same cycle, and you can use it for any new topic:

```
Concept to interview
CONCEPT -> WHY -> FLOW -> CODE -> RUN -> TRACE
   -> MODIFY / BREAK / FIX -> INTERNALS -> INTERVIEW QUESTIONS
```

**VERSION NOTE** MCP is a moving target. This document is accurate for the 2025-era protocol (initialize handshake, Streamable HTTP) and the Python SDK `mcp` 1.30.0. The study chat reported that the newest specification revision moves towards a stateless model with a discovery call instead of the initialize handshake. That could not be confirmed from inside this session, so before an interview check the current specification at modelcontextprotocol.io and treat the handshake described here as the widely deployed 2025 behaviour.

# 1. Introduction to MCP

## 1.1 What is MCP?

The **Model Context Protocol (MCP)** is an open standard introduced by Anthropic in November 2024. It defines a universal, vendor-neutral way for AI applications to connect to external data sources, tools and services. The usual analogy is **USB-C for AI integrations**: one standard plug that works everywhere, instead of a different cable for every device.

Before MCP, every team wrote a custom connector for each combination of AI application and external system. MCP replaces that tangle with one protocol that any compliant AI application and any compliant server can speak.

> **INTERVIEW** "MCP, the Model Context Protocol, is an open protocol for standardizing how AI applications connect to external data sources, tools and services. Instead of building a custom integration between every AI application and every system, an application talks to MCP servers that expose standardized capabilities such as tools and resources. MCP is a protocol, not an LLM framework like LangChain." The last sentence is what makes the answer stronger.

## 1.2 Why MCP was created

- AI assistants need context from live systems (databases, files, APIs) but there was no standard way to get it.
- Each team reinvented integration code, which caused fragmentation and security risk.
- A single protocol means one server works with every MCP-compliant host.
- It enables an ecosystem of reusable, shareable servers.

## 1.3 The integration problem MCP solves (the N x M problem)

Take a claims AI application. It needs these capabilities:

```
Capabilities of a claims AI
Claims AI Agent
     |
     +---- get_claim()
     +---- get_policy()
     +---- get_exposure()
     +---- create_reserve()
     +---- search_documents()
```

Without MCP you can implement each one directly. That works fine for one application:

```
One application, direct integrations
AI Agent
   |
   +---- Python function --> Claims API
   +---- Python function --> Policy API
   +---- Python function --> Exposure API
   +---- Python function --> Document DB
```

Now scale it. With **10 AI applications and 20 external systems** you can end up with up to 10 x 20 = 200 custom integrations, each with its own authentication, error handling and maintenance. That is the **N x M problem**, and it is the problem MCP was designed to standardise.

```
The MCP change: put the capability behind one server
Without MCP                          With MCP

AI App 1 ---- Claims API                              +-- Claims MCP Server
AI App 2 ---- Claims API             AI App 1 -MCP--|
AI App 3 ---- Claims API             AI App 2 -MCP--|    Tools
AI App 4 ---- Claims API             AI App 3 -MCP--|     get_claim, get_policy,
(each app re-implements it)          AI App 4 -MCP--+     get_exposure, create_reserve
                                                         Resources
                                                          claim documents, policy documents,
                                                          claim history
```

The MCP server exposes standardised capabilities; every application consumes them the same way.

## 1.4 MCP is a protocol, not an AI framework

This is the most important interview point in the whole topic. Three different things get confused:

| Thing | What it is | One-line description |
|---|---|---|
| LangChain / LangGraph | A framework | Helps you build LLM applications: prompts, chains, tools, agents, state |
| Function calling | A model/API capability | The model says "I want to call get_claim(claim_id='CLM123')" and your application executes it |
| MCP | A protocol | Standardises how an AI application talks to external capabilities |

```
Three different layers
LangChain      -> framework        (how you BUILD the LLM application)
Function call  -> model mechanism  (how the LLM ASKS for a tool)
MCP            -> protocol         (how the application REACHES external capabilities)
```

## 1.5 Why does the LLM need MCP at all?

The LLM cannot magically access a claims database, a policy system, SAP, a CRM, email or internal APIs. Something has to sit between the model and those systems.

```
Traditional vs MCP path from LLM to system
Traditional                          With MCP

LLM                                  LLM
 |                                    |
Application code                     Host (the AI application)
 |                                    |
Function                             MCP Client
 |                                    |
API                                  MCP Server
                                      |
                                     API / Database / System
```

MCP standardises the **middle** of that path.

## 1.6 An MCP server is not "a REST API", and not an LLM

- An MCP server is a logical **protocol server**. It speaks MCP over some transport (stdio or HTTP, see Section 3). Do not equate "MCP server" with "REST endpoint".
- An MCP server does **not** contain an LLM. It can simply expose enterprise capabilities. MCP Server is not LLM.

```
What an MCP server really is
MCP Server
   |
   +-- speaks the MCP protocol
   +-- exposes tools / resources / prompts
   +-- underneath: APIs, databases, files, Guidewire ...
```

## 1.7 Why MCP when LangGraph already has tools?

This was the first big doubt in the study chat: "We already had tools in LangGraph. Why did MCP become necessary?" It deserves a careful answer.

> **KEY IDEA** MCP did not invent tools. It standardised how tools and other capabilities are **exposed to and consumed by** AI applications.

### A LangGraph tool already works

```
Application-level tool
@tool
def get_claim(claim_id: str):
    return get_claim_from_database(claim_id)

tools = [get_claim]
llm_with_tools = llm.bind_tools(tools)
```

```
The tool lives inside the application
LangGraph application
   |
  LLM -> tool call -> ToolNode -> get_claim() -> Database
```

That is perfectly valid. The problem was never "we do not know how to create tools". The problem was: **how do we make these capabilities available, in a standard way, across many AI applications, frameworks and languages?**

### Ten applications, the same capability

Imagine Claims AI, Underwriting AI, Customer Service AI, Fraud AI, SIU AI, Developer AI, Document AI, Pricing AI, Policy AI and Analytics AI, all needing the claims, policy, customer and fraud systems. Without MCP each application builds its own `get_claim()` and its own `get_policy()`. They all work, but the integration logic is duplicated ten times.

### LangGraph tools belong to the application

When you write `@tool def get_claim(...)`, that tool is part of **your Python application**. Another application cannot ask yours, "give me the tools from your LangGraph app". There is no universal protocol for that.

## MCP changes who owns the capability

```
Ownership moves from each app to a shared server
              Claims MCP Server
             /        |        \
      Claims AI   Underwriting AI   Fraud AI
         |             |              |
     MCP Client    MCP Client     MCP Client
```

The tools are no longer implemented inside each AI application; they are exposed by a standardised MCP server.

## The analogy with REST

Functions already existed before REST. REST did not exist because "functions did not exist"; it gave applications a **standard way to talk to each other**. MCP is similar in spirit but designed for AI applications:

| Layer | Role |
|---|---|
| Python function | Application-level capability |
| REST API | Standardised application-to-application communication |
| MCP | Standardised AI application to capability communication |

## MCP is bigger than tools

Tools are only one MCP primitive. MCP also standardises **Resources** (readable data), **Prompts** (reusable instructions), **Sampling**, **Roots** and **Elicitation** (Section 4). So MCP is not "another way to define `def get_claim()`"; it is a standard for exposing different kinds of capability.

## "Why not just call REST APIs?"

You absolutely can: LangGraph, Python tool, REST API, claims system is a perfectly reasonable design, and MCP does not make REST obsolete. MCP becomes useful when you want a **standardised capability layer for AI clients**. The MCP server can hide whether the backend is PostgreSQL, REST, Guidewire, SOAP, Python or Java; the AI application only sees standard MCP capabilities.

## It also solves the framework problem

```
Many frameworks, one server
LangGraph ------+
CrewAI ---------+
Semantic Kernel +--> MCP Server --> Enterprise System
AutoGen --------+
Plain Python ---+
```

A capability built once is consumable by LangGraph, CrewAI, Semantic Kernel, AutoGen or plain Python, without rewriting the integration.

## What to memorise (and what not to)

Do not memorise "MCP is better than LangGraph tools". That is wrong, because they solve different layers.

| Technology | Question it answers |
|---|---|
| LangGraph tool | How does my agent execute a tool **inside my application**? |
| MCP | How can an AI application **discover and communicate** with external capabilities through a standard protocol? |

```
They work together
              LangGraph  (orchestration: state, nodes, routing)
                 |
                LLM -- decides to call get_claim
                 |
             MCP Client
                 |  MCP protocol
           Claims MCP Server
            /      |       \
       Policy   Claims    Fraud
        API      API       API
```

LangGraph stays the **orchestration layer**; MCP is the **standardised capability layer**. In the claims project: a Settlement Recommendation Agent needs policy details, loss assessment, fraud score and repair-vs-replacement. Without MCP it talks to SQL, APIs, Python functions and another agent directly. With MCP it uses one MCP client against a Claims MCP Server that exposes `get_policy_details()`, `get_loss_assessment()`, `get_fraud_score()` and `get_repair_decision()`.

> **KEY IDEA** Before MCP we had many ways to build tools; MCP standardises how AI applications discover, communicate with and use external capabilities.

## 1.8 Write once, reuse across AI applications (with code)

The follow-up question: "So n AI applications can use one MCP tool without rebuilding it? Show me in code."

```
Without MCP vs with MCP
Without MCP                              With MCP
Claims AI  -> get_claim()  (copy 1)                      Claims MCP Server
Fraud AI   -> get_claim()  (copy 2)   Claims AI  --+          |
UW AI      -> get_claim()  (copy 3)   Fraud AI   --+--> get_claim()  (ONE copy)
SIU AI     -> get_claim()  (copy 4)   UW AI      --+          |
                                                         Claims database
```

The server below is written once. Two completely separate applications then call it over Streamable HTTP. Neither application contains a `get_claim()` implementation.

```python
server (written ONCE): 104_http_server.py
from mcp.server.fastmcp import FastMCP

# Streamable HTTP: the server is a long-running HTTP service (default /mcp)
mcp = FastMCP("Claims HTTP Server", stateless_http=True)

CLAIMS = {
    "CLM001": {"claim_id": "CLM001", "policy_id": "POL1001", "status": "OPEN",
               "estimated_amount": 18500},
}


@mcp.tool()
def get_claim(claim_id: str) -> dict:
    """Retrieve claim details using a claim ID."""
    print(f"[MCP SERVER] get_claim() called with {claim_id}")   # safe on HTTP
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return {"error": f"Claim {claim_id} not found"}
    return claim


if __name__ == "__main__":
    mcp.run(transport="streamable-http")      # http://127.0.0.1:8000/mcp
```

```
Claims AI application: l04_http_client.py
import asyncio

from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

URL = "http://127.0.0.1:8000/mcp"


async def main(app_name: str = "Claims AI"):
    async with streamablehttp_client(URL) as (read, write, _get_session_id):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            print(f"[{app_name}] tools:", [t.name for t in tools.tools])

            result = await session.call_tool("get_claim", {"claim_id": "CLM001"})
            # a plain "-> dict" tool comes back as JSON text in result.content
            print(f"[{app_name}] received:", result.content[0].text)


if __name__ == "__main__":
    asyncio.run(main())
```

```
Fraud AI application: l01_fraud_ai.py
import asyncio

from l04_http_client import main

# A completely separate application (Fraud AI) reusing the SAME MCP server.
# There is no get_claim() implementation in this file.
asyncio.run(main("Fraud AI"))
```

Start the server in one terminal, then run each application in another:

```
Terminals
# Terminal 1
python l04_http_server.py

# Terminal 2
python l04_http_client.py        # Claims AI
python l01_fraud_ai.py           # Fraud AI
```

```
OUTPUT  two applications, one server
# --- Terminal 1: python l04_http_server.py  (server log) ---
INFO:     127.0.0.1:62576 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:62577 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:62578 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:62579 - "POST /mcp HTTP/1.1" 200 OK
[MCP SERVER] get_claim() called with CLM001

# --- Terminal 2: python l04_http_client.py ---
[Claims AI] tools: ['get_claim']
[Claims AI] received: {
  "claim_id": "CLM001",
  "policy_id": "POL1001",
  "status": "OPEN",
  "estimated_amount": 18500
}

# --- Terminal 2: python l01_fraud_ai.py ---
[Fraud AI] tools: ['get_claim']
[Fraud AI] received: {
  "claim_id": "CLM001",
```

```
  "policy_id": "POL1001",
  "status": "OPEN",
  "estimated_amount": 18500
}
```

**TESTED** Both applications received the same data from one implementation. The server log also shows the real HTTP conversation of one client: `POST /mcp` returns 200 (initialize), 202 Accepted (the `initialized` notification, which has no response), 200 (tools/list), 200 (tools/call).

### But a REST API could do the same

Yes. A Claims REST API would also remove the duplication. So MCP's value is **not** "multiple applications can reuse a function". The precise statement is:

**KEY IDEA** MCP provides a standardised protocol and capability model, designed for AI applications, to discover and interact with tools, resources and prompts.

A server can expose Tools (`get_claim`, `get_policy`, `create_reserve`, `check_fraud`), Resources (`claim://CLM001`, `policy://POL1001`) and Prompts (`fraud-investigation`, `claim-summary`). An AI application can **discover** what the server offers (`list_tools()` and so on) instead of having the list hard-coded.

**WATCH OUT** MCP does not make every tool automatically reusable by every application. The applications need MCP client support and appropriate authentication and authorisation, and the server must expose the capability.

## 1.9 MCP vs REST: the "AI-facing contract"

A common half-remembered idea: "an API returns a response in some other format, and via an MCP server we get it back in a format the AI understands". It is partly right and needs a correction.

Suppose a claims API returns:

```
Raw API response
{
  "claimNumber": "CLM123",
  "policy": { "id": "POL456" },
  "loss": { "type": "GLASS", "estimatedAmount": 4500 },
  "internalCode": "X7A91"
}
```

An LLM can read JSON, so nothing here is "un-understandable". But your AI application would have to know what the API means, which fields matter, how to call it, what arguments it needs, and how to expose it to the model. MCP adds an **AI-facing contract**. The MCP server adapts the messy enterprise response:

```
Adapter / translation layer
MCP Tool -> REST API -> raw response -> transform / validate / normalise -> MCP structured result
```

```
MCP tool as an adapter over a REST API
@mcp.tool()
def get_claim(claim_id: str) -> dict:
    response = requests.get(f"https://claims-api/claims/{claim_id}")
    data = response.json()
    return {
        "claim_id": data["claimNumber"],
        "policy_id": data["policy"]["id"],
        "loss_type": data["loss"]["type"],
        "estimated_amount": data["loss"]["estimatedAmount"],
    }
```

Now the AI-facing result is a clean, minimal object, and `internalCode` is never exposed.

> **WATCH OUT** MCP does **not** magically convert "REST format" into "AI language". There is no special "format the AI understands". MCP gives the application a standardised protocol and a structured capability interface; the MCP server developer decides how to transform the underlying response.

```
Where the integration work lives
WITHOUT MCP                          WITH MCP
AI Agent                             AI Agent
 |- knows the REST endpoint           |
 |- knows authentication             MCP Client
 |- knows the request format          |
 |- understands the response         MCP Tool / MCP Server
 |- transforms the data               |- calls the REST API
 v                                    |- transforms the response
REST API                              |- validates the data
                                      +- exposes a structured result
                                       v
                                      Enterprise system
```

## 1.10 MCP vs traditional approaches

| Approach | Problem | MCP solution |
|---|---|---|
| Custom plug-ins | Tied to one model or platform | Works with any MCP host |
| Function calling | No standard schema for discovery across apps | Standard capability negotiation and discovery |
| RAG pipelines | Read-only; no tool execution | Resources + Tools + Sampling |
| LangChain tools | Framework-specific, no transport standard | Language-agnostic JSON-RPC |

## 1.11 Key benefits

- **Standardisation**: one protocol, any language, any model.
- **Security**: servers run locally or in controlled environments; the model never receives raw credentials.
- **Composability**: a host can connect to many MCP servers at once.
- **Discoverability**: clients learn what a server can do at runtime.
- **Ecosystem**: community servers for GitHub, Postgres, Slack and more are immediately usable.

## 1.12 The three primitives to remember first

Do not try to memorise all six primitives yet. Start with these:

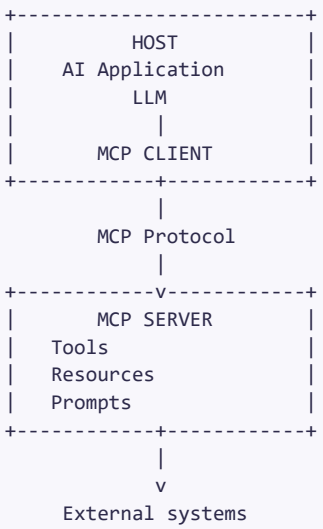
| MCP primitive | Think of it as |
|---|---|
| Resource | "Give me this data" (example: `claim://CLM123`) |
| Tool | "Do this action" (example: `create_reserve(claim_id, amount)`) |
| Prompt | "Use this reusable instruction" |

# 2. MCP Architecture

## 2.1 The three roles: Host, Client, Server

MCP uses a three-role architecture. Each role has a clear responsibility.

The architecture




| Role | What it is | Example |
|---|---|---|
| Host | The application the end user interacts with. It owns the LLM and orchestrates everything. | Claude Desktop, VS Code, your Claims AI, your Underwriting AI |
| Client | A thin protocol layer inside the host. Each client keeps exactly one connection to one server. | The MCP client SDK inside your application |
| Server | An independent process that exposes Resources, Tools and Prompts. It does not know which model will use its output. | A Claims MCP Server |

## 2.2 Host in depth

The Host is the actual AI application where the AI experience lives. It generally owns the LLM interaction and decides things like "the user wants to investigate this claim".

The host owns the LLM

```
User
  |
Claims AI Application      <- HOST
  |
LLM
```

## 2.3 MCP Client in depth

The MCP Client is **not** the same thing as the Host, which is the point most candidates get wrong. The client is the communication layer between the host and one MCP server.

Host contains the client

```
HOST
 |-- LLM
 +-- MCP CLIENT --(MCP protocol)--> MCP SERVER
```

## 2.4 MCP Server in depth

The server exposes capabilities. For the claims example:

```
Claims MCP Server
Claims MCP Server
  Tools      get_claim(), get_policy(), check_fraud(), create_reserve()
  Resources  claim://CLM123, policy://POL456
  Prompts    claim_summary, fraud_investigation
     |
     +--> Claims API, Policy API, Database, Guidewire
```

The server does not have to contain an LLM, and it talks to the enterprise systems on its own.

## 2.5 One host, many clients

A single host can run many clients at once, each connected to a different server. A claims AI may need a Claims MCP Server, a Policy MCP Server and a Fraud MCP Server.

```
One client per server connection
                 Claims AI Host
                       |
        +--------------+--------------+
        |              |              |
    Client 1       Client 2       Client 3
        |              |              |
  Claims Server  Policy Server  Fraud Server
```

Why not one client for everything? The MCP model treats the client as the **connection/session layer for one server**. Keeping one client per server isolates capabilities and connections.

> **TIP** This lets an assistant use a filesystem server, a database server and a web-search server all at the same time.

## 2.6 Communication flow (step by step)

| Step | Actor | Action |
|---|---|---|
| 1 | Host to Client | Start the client and open the transport connection to the server |
| 2 | Client to Server | Send `initialize` with protocol version and capabilities |
| 3 | Server to Client | Reply with its own capabilities (tools, resources, prompts) |
| 4 | Client to Host | Report the available capabilities |
| 5 | Host (LLM) | The model decides to call a tool; the host invokes the client |
| 6 | Client to Server | Send `tools/call` with the arguments |
| 7 | Server | Run the business logic (query a DB, call an API, read a file) |
| 8 | Server to Client | Return the result |
| 9 | Client to Host | Deliver the result into the LLM context |
| 10 | Host (LLM) | Generate the final answer for the user |

## 2.7 One complete request, traced

User says: **"Check claim CLM123 and tell me whether the policy covers the damage."**

```
The complete flow
User
 |
Host / AI application
 |
LLM -- decides: "I need claim information"
 |
MCP Client -- tools/call (get_claim, claim_id=CLM123)
 |
Claims MCP Server
 |
Claims system -> claim data
 |
MCP Server -> MCP Client -> LLM
 |
Final answer
```

For "Get me the details of claim CLM123" the steps are: the Host receives the request; the LLM decides it needs `get_claim` with `claim_id = CLM123`; the MCP Client sends the request; the server executes `get_claim()` against the Claims API and database; the result travels back (Database, Claims API, MCP Server, MCP Client, Host/LLM); the LLM writes "Claim CLM123 is an open claim involving windshield damage...".

> **NOTE** The newer protocol revisions change how capabilities are discovered (see the version note in the front matter). Do not memorise the initialize handshake as the only possible behaviour; it is the widely deployed 2025 model.

## 2.8 Terminology traps (do not say these in an interview)

| Wrong statement | Why it is wrong | Correct |
|---|---|---|
| "The MCP Client is the AI." | The client is only the connection layer | The LLM lives in the Host |
| "The MCP Server is the database." | The server is a protocol endpoint in front of systems | The server exposes capabilities that may use a database |
| "The Host is the MCP Server." | Different roles | Host = AI app, Server = capability provider |

Memorise it as: **Host = AI application/environment. Client = communication component connecting the Host to a Server. Server = provider of MCP capabilities.**

## 2.9 JSON-RPC 2.0 at a glance

All MCP messages are **JSON-RPC 2.0** envelopes. There are three message types:

- **Request**: expects a response. It has an `id`, a `method` and optional `params`.
- **Response**: the reply to a request. It has the same `id` and either `result` or `error`.
- **Notification**: fire-and-forget. It has a `method` and params but **no** `id`.

```
The four message shapes
// Request
{ "jsonrpc": "2.0", "id": 1, "method": "tools/call",
  "params": { "name": "get_weather", "arguments": { "city": "London" } } }

// Success response
{ "jsonrpc": "2.0", "id": 1,
  "result": { "content": [{ "type": "text", "text": "18C, partly cloudy" }] } }
```

```
// Error response
{ "jsonrpc": "2.0", "id": 1,
  "error": { "code": -32601, "message": "Method not found" } }

// Notification (no id)
{ "jsonrpc": "2.0", "method": "notifications/progress",
  "params": { "progressToken": "abc", "progress": 50 } }
```

The full treatment, with working code, is in Section 7.

## 2.10 Host, Client and Server compared with LangGraph

You already know the LangGraph tool loop. MCP sits at a different layer.

```
LangGraph tool loop (local tools)
LangGraph
   |
  LLM -> ToolNode -> get_claim() / get_policy() / calculate_reserve()
```

```
With MCP
LangGraph Host
   |
MCP Client -> MCP Server -> get_claim() / get_policy() / create_reserve()
```

| LangGraph handles | MCP handles |
|---|---|
| State, nodes, edges | The standardised communication with external capabilities |
| Conditional routing | Capability discovery |
| Reducers | Tool/resource/prompt exposure |
| Agent loops | Transport and message format |

They complement each other; they do not compete.

## 2.11 Interview questions on the architecture

### Q. What is the difference between an MCP Host, Client and Server?

The Host is the AI application that owns the user interaction and the LLM. The MCP Client is the component inside the Host that communicates with an MCP Server. The MCP Server exposes capabilities such as tools, resources and prompts and can connect to underlying enterprise systems. A Host can hold multiple client connections to different servers.

### Q. Can a single host talk to several MCP servers?

Yes. The host runs one client per server connection. For example a claims assistant can have a client for the Claims server, one for the Policy server and one for the Fraud server at the same time.

### Q. Does an MCP server need an LLM?

No. The server only exposes capabilities. The host owns the LLM and decides when to call them.

### Q. Can an MCP server connect to another MCP server?

Yes. A server can act as a client to another server, enabling composition and agentic architectures where one server orchestrates others.

# 3. Transport Layer

## 3.1 Protocol vs transport: the key separation

Two ideas must be kept apart:

> **KEY IDEA** **Protocol** = what the messages mean. **Transport** = how those messages travel.

MCP messages are JSON-RPC. The transport is the mechanism that carries them between client and server. The same messages can travel over different transports depending on where the server runs.

```
Protocol rides on a transport
                    MCP protocol: JSON-RPC messages
                                  |
+--------------+   Transport   +------------------+
| AI Host      |               | MCP Server       |
|  LLM         |               |  get_claim()     |
|  MCP Client -+---------------+> get_policy()    |
|              | stdio / HTTP  |  create_reserve()|
+--------------+               +------------------+
```

| Concept | Responsibility | Example |
|---|---|---|
| MCP protocol | Defines how components communicate | `tools/call` |
| JSON-RPC | Defines the message structure | `method`, `params`, `id` |
| Transport | Carries messages between components | stdio, Streamable HTTP |
| MCP tool | Performs a capability or action | `get_claim()` |

The transport does not decide whether a claim is valid. It only carries the request to the component that handles it.

```
The full layering (memorise this)
AI APPLICATION
      |
    LLM
      |
 MCP Client
      |
+-------------+
|  Transport  |   stdio / HTTP / Streamable HTTP
+-------------+
      |
 MCP JSON-RPC
      |
+-------------+
| MCP Server  |   Tools / Resources / Prompts
+-------------+
      |
 Enterprise systems: Guidewire, DB, REST
```

## 3.2 Why we need more than one transport

**Scenario A, local application.** An AI application and a claims MCP server run on the same laptop. They do not need an HTTP port; they can communicate through operating-system process streams (stdio).

**Scenario B, remote deployment.** The MCP server runs on an Azure VM while the application runs elsewhere. They communicate over an HTTP-based transport.

> **KEY IDEA** stdio is for local process communication; HTTP-based transports are for talking to a server over a network.

## 3.3 stdio transport

**stdio** means standard input/output. Instead of opening an HTTP server, the host starts the MCP server as a **local child process**.

| Stream | Meaning | Used for |
|---|---|---|
| stdin | Standard input | Client to server messages |
| stdout | Standard output | Server to client messages |
| stderr | Standard error | Diagnostic logs |

```
stdio
Claims AI Host
      |
      | starts a process
      v
python claims_mcp_server.py
      |
      +-- stdin   <- requests
      +-- stdout  -> responses
      +-- stderr  -> logs
```

Messages are newline-delimited JSON; the SDK handles the framing. Properties:

- Simple: no network port and no firewall rules.
- Secure by isolation: the process boundary is the security boundary.
- Used by Claude Desktop for local integrations (see Section 13 for the config file).
- Single client only.

### Why stdio does not need a port

With a normal REST service you would run `app.run(port=8000)` and the client would connect to `localhost:8000`. With stdio the operating system gives the parent and child process pipes (stdin/stdout). There is no TCP socket, so there is no `localhost:8000`:

```
Pipes, not sockets
Host --> stdin  --> MCP Server
Host <-- stdout <-- MCP Server
```

### What happens when the client launches the server

```
Launch flow
1. python client.py starts
2. Client creates server parameters (command + args)
3. MCP client launches server.py as a subprocess
4. stdin/stdout communication is established
5. MCP session is initialised
6. Client discovers tools (tools/list)
7. Client invokes get_claim()  (tools/call)
8. Server returns the result
```

The client never connects to `localhost:8000`; it already owns a channel to the process it launched.

## 3.4 stdio in code (tested)

Server:

```
l04_stdio_server.py
import sys
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Claims Server")

CLAIMS = {
    "CLM001": {"claim_id": "CLM001", "policy_id": "POL1001", "status": "OPEN",
               "loss_type": "Windshield Damage", "estimated_amount": 18500},
    "CLM002": {"claim_id": "CLM002", "policy_id": "POL1002", "status": "CLOSED",
               "loss_type": "Collision", "estimated_amount": 75000},
}


@mcp.tool()
def get_claim(claim_id: str) -> dict:
    """Retrieve claim details using a claim ID."""
    # stdout is reserved for the protocol on stdio, so log to stderr
    print(f"[SERVER] get_claim called with {claim_id}", file=sys.stderr)
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return {"error": f"Claim {claim_id} not found"}
    return claim


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Client:

```
l04_stdio_client.py
import asyncio
import sys
from pathlib import Path

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER = Path(__file__).with_name("l04_stdio_server.py")


async def main():
    # 1. How should the server process be launched?
    params = StdioServerParameters(command=sys.executable, args=[str(SERVER)])

    # 2. Launch it and get the two stdio streams
    async with stdio_client(params) as (read, write):
        # 3. Wrap the streams in an MCP session
        async with ClientSession(read, write) as session:
            await session.initialize()                      # handshake

            tools = await session.list_tools()              # tools/list
            print("Available tools:")
            for tool in tools.tools:
                print(f"- {tool.name}: {tool.description}")

            result = await session.call_tool(               # tools/call
                "get_claim", {"claim_id": "CLM001"})
            print("\nisError:", result.isError)
            print("structuredContent:", result.structuredContent)
            for item in result.content:
                if hasattr(item, "text"):
                    print("text:", item.text)
```

```
asyncio.run(main())
```

Run it. You only start the client; it launches the server itself:

```
Run
python l04_stdio_client.py
```

```
OUTPUT  stdio client run
Available tools:
- get_claim: Retrieve claim details using a claim ID.

isError: False
structuredContent: None
text: {
  "claim_id": "CLM001",
  "policy_id": "POL1001",
  "status": "OPEN",
  "loss_type": "Windshield Damage",
  "estimated_amount": 18500
}
```

**TESTED** This produced exactly the output above. Because the tool returns a plain `dict`, the result arrives as **JSON text in** `result.content` and `result.structuredContent` is `None`. Use a typed return (a Pydantic model or TypedDict) when you want structured content.

**WATCH OUT** **Never** `print()` **to stdout in a stdio server.** stdout carries the protocol; a stray `print("Server started")` corrupts the stream. Log with `print(..., file=sys.stderr)` or the `logging` module configured for stderr.

## Tracing the client code line by line

**A. Define the server process.** `StdioServerParameters(command=sys.executable, args=[str(SERVER)])`. `sys.executable` is the interpreter running the client, so the server uses the same virtual environment. Conceptually it runs `python server.py`.

**B. Establish the transport.** `async with stdio_client(params) as (read, write)` launches the subprocess and returns two streams: `write` (client to server) and `read` (server to client). The SDK handles framing.

**C. Create the session.** `ClientSession(read, write)` gives high-level methods instead of hand-built JSON-RPC.

**D. Discover tools.** `await session.list_tools()` corresponds to `tools/list`.

**E. Execute the tool.** `await session.call_tool("get_claim", {...})` corresponds to `tools/call`.

**F. Read the result.** The client receives an SDK result object, not the server's Python dict. A returned dictionary is exposed as serialised text (or structured content, depending on the SDK and return type).

## Debugging: the client cannot find server.py

Symptom: `python: can't open file 'server.py'`. The cause is the working directory: the client was started from a different folder than the one containing `server.py`. Fix it by changing directory, or better, build the path relative to the client file so it works from anywhere:

```
Path relative to the client file
from pathlib import Path

server_path = Path(__file__).with_name("l04_stdio_server.py")
params = StdioServerParameters(command=sys.executable, args=[str(server_path)])
```

## 3.5 HTTP transport and Streamable HTTP

Now imagine the server runs elsewhere:

```
Remote server
AI Application --(Internet / network)--> https://company.com/mcp --> MCP Server
```

Unlike stdio, the client does **not** launch the server. The server is already running and listening for HTTP requests. This is much closer to what you have done with FastAPI.

**Streamable HTTP** (introduced in the 2025 revisions) is the modern HTTP transport. It uses a **single HTTP endpoint**, commonly `/mcp`:

- The client POSTs MCP messages to the endpoint.
- The server answers either with a single JSON body or by streaming the response (SSE-style) when it needs to send several messages.
- A session id header can tie requests together; a **stateless** mode avoids server-side sessions.

**WATCH OUT** A route called `/mcp` does not by itself make a server an MCP server. A normal FastAPI endpoint that accepts arbitrary JSON is **not** automatically MCP; the server must implement the MCP protocol over that transport.

### Streamable HTTP in code (tested)

Server:

```
l04_http_server.py
from mcp.server.fastmcp import FastMCP

# Streamable HTTP: the server is a long-running HTTP service (default /mcp)
mcp = FastMCP("Claims HTTP Server", stateless_http=True)

CLAIMS = {
    "CLM001": {"claim_id": "CLM001", "policy_id": "POL1001", "status": "OPEN",
               "estimated_amount": 18500},
}


@mcp.tool()
def get_claim(claim_id: str) -> dict:
    """Retrieve claim details using a claim ID."""
    print(f"[MCP SERVER] get_claim() called with {claim_id}")   # safe on HTTP
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return {"error": f"Claim {claim_id} not found"}
    return claim


if __name__ == "__main__":
    mcp.run(transport="streamable-http")      # http://127.0.0.1:8000/mcp
```

Client:

```
l04_http_client.py
import asyncio

from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

URL = "http://127.0.0.1:8000/mcp"


async def main(app_name: str = "Claims AI"):
    async with streamablehttp_client(URL) as (read, write, _get_session_id):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            print(f"[{app_name}] tools:", [t.name for t in tools.tools])

            result = await session.call_tool("get_claim", {"claim_id": "CLM001"})
            # a plain "-> dict" tool comes back as JSON text in result.content
            print(f"[{app_name}] received:", result.content[0].text)


if __name__ == "__main__":
    asyncio.run(main())
```

Run the server in Terminal 1, the client in Terminal 2:

```
Run
# Terminal 1
python l04_http_server.py        # listens on http://127.0.0.1:8000/mcp

# Terminal 2
python l04_http_client.py
```

```
OUTPUT  HTTP server log and clients
# --- Terminal 1: python l04_http_server.py  (server log) ---
INFO:     127.0.0.1:62576 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:62577 - "POST /mcp HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:62578 - "POST /mcp HTTP/1.1" 200 OK
INFO:     127.0.0.1:62579 - "POST /mcp HTTP/1.1" 200 OK
[MCP SERVER] get_claim() called with CLM001

# --- Terminal 2: python l04_http_client.py ---
[Claims AI] tools: ['get_claim']
[Claims AI] received: {
  "claim_id": "CLM001",
  "policy_id": "POL1001",
  "status": "OPEN",
  "estimated_amount": 18500
}

# --- Terminal 2: python l01_fraud_ai.py ---
[Fraud AI] tools: ['get_claim']
[Fraud AI] received: {
  "claim_id": "CLM001",
  "policy_id": "POL1001",
  "status": "OPEN",
  "estimated_amount": 18500
}
```

**TESTED** The log lines show the real HTTP exchange (200, 202, 200, 200 = initialize, initialized notification, tools/list, tools/call). `print()` is safe here because the protocol travels over HTTP, not stdout.

**VERSION NOTE** `stateless_http=True` is an SDK option controlling how the HTTP server handles sessions. It does not turn arbitrary FastAPI routes into MCP tools. Newer SDK releases rename some classes (the study chat mentioned `MCPServer` and a `Client` that takes a URL); these examples use the `mcp` 1.30.0 API (`FastMCP`, `ClientSession`, `streamablehttp_client`). Do not mix examples from different SDK generations; check the installed version and its documentation.

Because the server runs independently you can restart the client without restarting the server, and several different applications can connect to it at once (Section 1.8).

## 3.6 HTTP+SSE (legacy)

**SSE** (Server-Sent Events) is a web technology that lets a server stream events to a client. The earlier MCP "HTTP+SSE" transport used two endpoints: the client opened `GET /sse` as a persistent stream for server push and POSTed requests to a separate `/message` endpoint. Responses came back down the SSE stream.

**NOTE** HTTP+SSE is a **legacy** pattern kept for compatibility with older servers and clients. For new work use Streamable HTTP, or stdio for local servers. Do not confuse the two ideas: SSE is a web technology; "MCP HTTP+SSE" is one particular way of carrying MCP messages.

## 3.7 Choosing a transport

| Criterion | stdio | HTTP (Streamable HTTP) |
|---|---|---|
| Server location | Usually local | Local or remote |
| Who starts the server | The client launches it as a subprocess | Already running; the client connects to a URL |
| Network required | No | Yes |
| Separate port | No | Usually yes |
| Communication | stdin/stdout pipes | HTTP requests (optionally streamed) |
| Multi-client | No | Yes |
| Security model | OS process isolation | Needs TLS plus authentication |
| Typical use | Desktop tools, IDE plugins, local dev | SaaS, shared team servers, FastAPI deployments |
| Main concern | Process config, keeping stdout clean | Network, auth, HTTP configuration |

If the server is unavailable: with stdio the launch fails or the pipe closes; with HTTP the connection is refused or times out. Handle both (Section 10).

## 3.8 Internals: how the two paths differ

```
stdio path
Client Python process
        | starts a subprocess
        v
Server Python process  (stdin / stdout)
```

```
Streamable HTTP path
Client --> HTTPS --> Load balancer / reverse proxy --> MCP HTTP endpoint --> tool execution
```

The HTTP path can reuse existing networking and security infrastructure (gateways, TLS termination, WAF). It also needs careful handling of authentication, authorisation, TLS, timeouts and access control.

> **WATCH OUT** **Transport is not security.** Choosing HTTP or stdio does not by itself make your tools safe.

## 3.9 The transport knows nothing about your business system

Consider a policy-coverage MCP server in front of Guidewire:

**Responsibilities stay separate**

```
LLM: "I need coverage for POL123"
 -> MCP Client -> tools/call (gw_get_policy_coverages)
 -> [transport carries the message]
 -> MCP Server tool -> Guidewire API
 -> server normalises the response -> tool result -> LLM
```

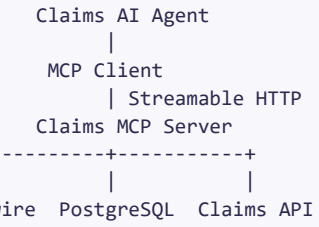
Transport carries the MCP message; the tool does the business operation. That separation keeps the architecture maintainable.

An MCP server can use **REST APIs internally**; this is a very common design:

**Tool calling an enterprise REST API**

```python
@mcp.tool()
def get_policy(policy_id: str):
    response = requests.get(f"https://guidewire/api/policies/{policy_id}")
    data = response.json()
    return {
        "policy_id": data["policyNumber"],
        "status": data["status"],
        "coverage_limit": data["coverageLimit"],
    }
```

**Enterprise claims architecture over Streamable HTTP**




## 3.10 Interview questions on transports

### Q. What is an MCP transport?

It defines how protocol messages travel between an MCP client and server. Examples are stdio for local process communication and Streamable HTTP for HTTP-based communication.

### Q. Why does stdio not require an HTTP port?

The client communicates with the server process through its standard input and output streams provided by the operating system; no network listener is needed.

### Q. What is the difference between stdio and Streamable HTTP?

stdio uses process streams and is typically used for local integrations, where the client launches the server as a subprocess. Streamable HTTP communicates over HTTP and supports remote deployments, with the client connecting to an already-running endpoint.

### Q. Is HTTP itself MCP?

No. HTTP is a transport. MCP is the protocol that defines how a client and server communicate and how tools, resources and prompts are represented. The same MCP semantics can run over stdio or HTTP.

### Q. Is an MCP server always a FastAPI application?

No. MCP servers can be written in many languages. FastAPI can be used with MCP, but an HTTP endpoint is not automatically an MCP server; it has to implement the MCP protocol.

### Q. Can an MCP server use REST APIs internally?

Yes. Tools can call REST APIs, databases, GraphQL or other systems. The transport carries MCP messages while the tool implements the business operation.

### Q. Which transport for an enterprise claims agent?

For an agent talking to a centrally hosted MCP service, Streamable HTTP. For a local desktop integration where the host launches the process, stdio. The choice depends on deployment, access control, networking and operations.

## 3.11 Exercises

- Add a second tool `get_claim_status(claim_id)` returning `{"claim_id": "CLM001", "status": "OPEN"}`; test it through both clients.
- Run the stdio server, then the HTTP server. Observe: does the client launch the server? Does the server listen on a port? Which command starts it? What happens if the server is down?
- Call `get_claim` with `CLM999` and decide whether to return a business-error object or raise a tool execution error.
- Production thinking: you deploy on an Azure VM. List what you must add: HTTPS, authentication, authorisation, firewall rules, logging and monitoring, DB connection management, request timeouts. None of these come from the transport.

# 4. Core Primitives

## 4.1 The six primitives at a glance

MCP defines six core abstractions. Three are exposed by **servers** (Resources, Tools, Prompts) and three are offered by **clients/hosts** to servers (Sampling, Roots, Elicitation). Start with the first three.

| Primitive | Exposed by | One-line meaning | Who decides to use it |
|---|---|---|---|
| Resources | Server | Readable data identified by a URI | The application / host |
| Tools | Server | Callable actions with a JSON-Schema input | The model (LLM) |
| Prompts | Server | Reusable, parameterised message templates | The user / application |
| Sampling | Client | Server asks the host to run an LLM completion | Host, with user approval |
| Roots | Client | URIs that scope where the server should work | Client |
| Elicitation | Client | Server asks the end user for more input | Host UI |

```
Server-side primitives
                  MCP SERVER
                      |
          +-----------+-----------+
          |           |           |
        TOOLS      RESOURCES    PROMPTS
          |           |           |
       Actions       Data     Instructions
          +-----------+-----------+
                      |
                MCP CLIENT -> AI HOST -> LLM
```

> **KEY IDEA** The host application decides how to combine these capabilities. MCP does not automatically run every tool, read every resource or send every prompt to an LLM.

## 4.2 Resources

### 4.2.1 What a Resource is

An MCP **Resource** exposes data or context that a client can read. It is read-oriented and identified by a **URI**. The nearest REST analogy is a GET endpoint.

- **Static resources**: a fixed URI, always available (example: `file:///config.json`, `reference://severity-buffers`).
- **Dynamic resources (URI templates)**: a URI with parameters (example: `claims://{claim_id}`).
- **Contents** can be text (UTF-8) or binary (base64) with a MIME type.
- **Subscriptions**: a client can subscribe to changes on a resource and receive notifications.

The same claim could be exposed as a tool `get_claim("CLM001")` or as a resource `claims://CLM001`:

```
Tool vs resource access
Tool                              Resource
Client -- get_claim("CLM001")     Client -- read claims://CLM001
  -> MCP Server                     -> MCP Server
  -> execute tool                   -> resolve the URI
  -> return result                  -> return contents
```

### 4.2.2 Why Resources exist if Tools can return data

MCP offers different abstractions for different purposes, and they are **not mutually exclusive**; one server can expose both.

| Requirement | Possible abstraction |
|---|---|
| Retrieve a claim record | Tool or Resource |
| Retrieve policy information | Tool or Resource |
| Calculate a recommended reserve | Tool |
| Create a reserve | Tool |
| Expose a claim document for reading | Resource |
| Expose reference data to an AI application | Resource |

```
One server, both primitives
Claims MCP Server
  Tools      calculate_reserve(), create_reserve()
  Resources  claims://CLM001, claims://CLM002, policies://POL1001
```

A Resource gives the client a way to access identifiable data through a URI. A Tool exposes a callable capability. A Resource is not automatically better just because the operation returns data: a Tool is the better abstraction when retrieval needs arguments, business logic, authorisation decisions or a specialised operation.

### 4.2.3 Understanding resource URIs

**Anatomy**

```
claims://CLM001
 |        |
 |        +-- identifier of the record
 +----------- URI scheme (your own, understood by the server)
```

The URI identifies a resource the server knows how to resolve. It is not necessarily a public website. `claims://CLM001` might resolve to `CLAIMS["CLM001"]`; in production it would go through a repository to PostgreSQL. The URI is the AI-facing identifier; the backend can change.

### 4.2.4 Flow of a resources/read

**Reading a resource**

```
USER -> AI APPLICATION -> MCP CLIENT
         resources/read  uri = claims://CLM001
              -> MCP SERVER: resolve URI -> retrieve data -> return contents
              -> MCP CLIENT -> AI APPLICATION
```

**WATCH OUT** Reading a resource does **not** put it in the model's context. The host decides which resources to read and how to include their contents in the LLM request.

### 4.2.5 Code: a server with resources, a static resource and a tool (tested)

**106_server.py**

```python
import json
import sys
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Claims Resource Server")

CLAIMS = {
    "CLM001": {"claim_id": "CLM001", "policy_id": "POL1001", "status": "OPEN",
               "loss_type": "Windshield Damage", "estimated_amount": 18500},
    "CLM002": {"claim_id": "CLM002", "policy_id": "POL1002", "status": "CLOSED",
               "loss_type": "Collision", "estimated_amount": 75000},
}
POLICIES = {
    "POL1001": {"policy_id": "POL1001", "status": "ACTIVE",
                "coverage_limit": 500000, "deductible": 1000},
}


# Resource TEMPLATE: the {claim_id} part is a variable in the URI
@mcp.resource("claims://{claim_id}")
def read_claim_resource(claim_id: str) -> str:
    """Read a claim record using a claims:// URI."""
    print(f"[SERVER] resources/read claims://{claim_id}", file=sys.stderr)
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return json.dumps({"error": "CLAIM_NOT_FOUND",
                           "message": f"Claim {claim_id} does not exist."})
    return json.dumps(claim, indent=2)


@mcp.resource("policies://{policy_id}")
def read_policy_resource(policy_id: str) -> str:
    """Read a policy record using a policies:// URI."""
    policy = POLICIES.get(policy_id)
    if policy is None:
        return json.dumps({"error": "POLICY_NOT_FOUND",
                           "message": f"Policy {policy_id} does not exist."})
```

```
    return json.dumps(policy, indent=2)


# STATIC resource: a fixed URI with no variables
@mcp.resource("reference://severity-buffers", mime_type="application/json")
def severity_buffers() -> str:
    """Reference data: severity buffers used by the reserve calculation."""
    return json.dumps({"Critical": 0.20, "High": 0.15, "Medium": 0.10, "Low": 0.05})


# A Tool living next to the resources, to contrast the two
@mcp.tool()
def calculate_reserve(claim_id: str, severity_buffer: float = 0.10) -> dict:
    """Calculate a simplified recommended reserve for a claim."""
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return {"error": "CLAIM_NOT_FOUND"}
    return {"claim_id": claim_id, "estimated_amount": claim["estimated_amount"],
            "severity_buffer": severity_buffer,
            "recommended_reserve": round(claim["estimated_amount"] * (1 + severity_buffer), 2)}


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

**l06_client.py**

```
import asyncio
import sys
from pathlib import Path

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER = Path(__file__).with_name("l06_server.py")


async def main():
    params = StdioServerParameters(command=sys.executable, args=[str(SERVER)])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # resources/list -> STATIC resources only
            resources = await session.list_resources()
            print("========== resources/list (static) ==========")
            for r in resources.resources:
                print(f"- {r.name}: {r.uri}")

            # resources/templates/list -> URI TEMPLATES
            templates = await session.list_resource_templates()
            print("\n========== resources/templates/list ==========")
            for t in templates.resourceTemplates:
                print(f"- {t.name}: {t.uriTemplate}")

            # resources/read
            print("\n========== resources/read claims://CLM001 ==========")
            res = await session.read_resource("claims://CLM001")
            for item in res.contents:
                print(item.text)

            print("\n========== resources/read policies://POL1001 ==========")
            res = await session.read_resource("policies://POL1001")
            print(res.contents[0].text)

            print("\n========== resources/read reference://severity-buffers ==========")
            res = await session.read_resource("reference://severity-buffers")
            print(res.contents[0].mimeType, res.contents[0].text)
```

```
            # Contrast: a TOOL call
            print("\n========== tools/call calculate_reserve ==========")
            tool_result = await session.call_tool(
                "calculate_reserve", {"claim_id": "CLM001", "severity_buffer": 0.10})
            print(tool_result.content[0].text)

            # Unknown resource
            print("\n========== resources/read claims://CLM999 ==========")
            res = await session.read_resource("claims://CLM999")
            print(res.contents[0].text)


asyncio.run(main())
```

```
OUTPUT  python l06_client.py
========== resources/list (static) ==========
- severity_buffers: reference://severity-buffers

========== resources/templates/list ==========
- read_claim_resource: claims://{claim_id}
- read_policy_resource: policies://{policy_id}

========== resources/read claims://CLM001 ==========
{
  "claim_id": "CLM001",
  "policy_id": "POL1001",
  "status": "OPEN",
  "loss_type": "Windshield Damage",
  "estimated_amount": 18500
}

========== resources/read policies://POL1001 ==========
{
  "policy_id": "POL1001",
  "status": "ACTIVE",
  "coverage_limit": 500000,
  "deductible": 1000
}

========== resources/read reference://severity-buffers ==========
application/json {"Critical": 0.2, "High": 0.15, "Medium": 0.1, "Low": 0.05}

========== tools/call calculate_reserve ==========
{
  "claim_id": "CLM001",
  "estimated_amount": 18500,
  "severity_buffer": 0.1,
  "recommended_reserve": 20350.0
}

========== resources/read claims://CLM999 ==========
{"error": "CLAIM_NOT_FOUND", "message": "Claim CLM999 does not exist."}
```

**TESTED** Findings from the real run: (1) `resources/list` returned **only the static resource**. The URI templates `claims://{claim_id}` and `policies://{policy_id}` appear under a **different call**, `resources/templates/list` (`session.list_resource_templates()`). (2) The static resource carried the MIME type `application/json`. (3) An unknown claim still returns normally with an error object, because our handler returns JSON text rather than raising.

### 4.2.6 Understanding the decorator

`@mcp.resource("claims://{claim_id}")` declares a URI template. `{claim_id}` is a variable: reading `claims://CLM001` calls `read_claim_resource("CLM001")`. We return `json.dumps(claim)` so the content is text that is easy to inspect; the result is carried as **content**, not as a Python object crossing the process boundary. Resource and prompt handlers also execute server-side Python; the difference between primitives is the **interface** they expose, not whether code runs.

### 4.2.7 Trace: Resource vs Tool side by side

```
Same data, different interface
RESOURCE  claims://CLM001   -> retrieve claim data -> return contents
TOOL      calculate_reserve -> validate inputs -> calculate -> return result
```

Both can return data. Their intended roles differ: the resource reads claim data; the tool performs a calculation.

### 4.2.8 Resource versus Tool: the interview table

| Feature | Resource | Tool |
|---|---|---|
| Main purpose | Expose data or context | Expose a callable capability |
| Identifier | URI | Tool name |
| Example | `claims://CLM001` | `calculate_reserve(...)` |
| Protocol operation | `resources/read` | `tools/call` |
| Can return data? | Yes | Yes |
| Business logic | A handler can retrieve or prepare data | A tool can execute a business operation |
| Typical use | Read claim documents or records | Calculate reserves, verify coverage, create records |

One subtle point: the distinction is **not** simply "returns data or not". Both return data. It is about the interface and intended role.

### 4.2.9 Connection to RAG

A treaty document could be exposed as a resource `treaty://TRT-2026-001` (access to identified content) or as a tool `search_treaty(treaty_id, query)` (a search operation with arguments). A RAG system then feeds retrieved content to an LLM. MCP can provide the interface to such capabilities, but MCP itself is not a vector database or a RAG algorithm.

### 4.2.10 Production considerations for resources

- **Authorisation**: not every user may read every claim; the handler or service must enforce access control.
- **Freshness**: is the data live, cached or versioned?
- **Large documents**: do not inject a huge document into the context; retrieve relevant portions.
- **Sensitive data**: return only authorised and necessary fields.
- **Errors**: define a consistent policy for missing resources, backend failure and authorisation failure (our demo returns an error object as text).

### 4.2.11 Interview questions on resources

#### Q. What is an MCP Resource?

A Resource exposes data or context through a URI. A client can discover resources and read their contents using the relevant MCP operations.

#### Q. What is the difference between a Resource and a Tool?

A Resource is a URI-identified source of readable data or context; a Tool is a callable capability with defined inputs and behaviour. Both can return data, but their interfaces and intended roles differ.

#### Q. What does resources/read do?

It requests the contents of the resource identified by a URI.

#### Q. Does a Resource automatically get added to the LLM's context?

No. The host application controls whether to read a resource and how to include its contents in the model request.

#### Q. Can a Resource read from PostgreSQL?

Yes. The handler can query a repository or database and expose the result through a URI, subject to validation and access control.

#### Q. Should you always use a Resource instead of a Tool when retrieving data?

No. A Tool may be better when retrieval needs complex parameters, business logic or a specific operation. A Resource suits identifiable content accessed through a URI.

#### Q. What is a URI template?

A pattern (RFC 6570 style) for dynamic resource URIs, for example `db://table/{tableName}/row/{id}`. Clients fill in the parameters to address a specific resource.

### 4.2.12 Practice

- Add a tool `verify_policy_status(policy_id)` returning `{"policy_id": "POL1001", "is_active": true}`.
- Explain why `policies://POL1001` and `verify_policy_status("POL1001")` are different interfaces.
- Add error handling for an unknown policy id.

## 4.3 Tools

**Tools** are callable functions that can have side effects; they are the MCP equivalent of actions (POST). The **LLM decides when to call a tool**; the host executes it.

- A tool is defined by a `name`, a `description` and an `inputSchema` (JSON Schema).
- The **description is critical**: the LLM reads it to decide when to call the tool. Clear descriptions make tool selection easier.
- Results are a list of content items (text, image, embedded resource).
- `isError: true` in a result signals a tool-level failure without failing the RPC.

### What the SDK derives from a Python function

```
A function becomes a tool
@mcp.tool()
def get_claim(claim_id: str) -> dict:
    """Get claim details using a claim ID."""
```

| Source in the function | Becomes |
|---|---|
| Function name `get_claim` | Tool name |
| Docstring | Tool description |
| Parameter `claim_id: str` | `inputSchema` property of type string, marked required |
| Default value (`severity_buffer: float = 0.10`) | Property with a default, not required |

```
What tools/list actually returns for calculate_reserve (from the tested run)
{
  "properties": {
    "claim_id": {"title": "Claim Id", "type": "string"},
    "severity_buffer": {"default": 0.1, "title": "Severity Buffer", "type": "number"}
  },
  "required": ["claim_id"],
  "title": "calculate_reserveArguments",
  "type": "object"
}
```

That machine-readable metadata is the key difference between `get_claim("CLM001")` as a Python call and an MCP tool: the client can **discover** the name, the description and the input contract. Building complete servers and clients is in Sections 5 and 6; the call lifecycle is in Section 7.6.

### Three kinds of failure (tested behaviour)

| Situation | What the client sees (mcp 1.30.0) | Retry? |
|---|---|---|
| Business error you return as data (`{"error": "CLAIM_NOT_FOUND"}`) | `isError=False`; the error is inside the JSON content | No, fix the input |
| Invalid argument type (schema validation) | `isError=True`; text starts with `Error executing tool ...` | No |
| Unknown tool name | `isError=True`; text `Unknown tool: ...` | No |
| Server crash, timeout, closed connection | Python exception or timeout in the client | Maybe (Section 10) |

> **WATCH OUT** Because a returned `{"error": ...}` dict has `isError=False`, a client that only checks `isError` will treat a business failure as success. Check the payload too, or raise an exception in the tool when you want `isError=True`.

## 4.4 Prompts

### 4.4.1 What an MCP Prompt is

**Prompts** are reusable, parameterised message templates that a server exposes. They let a server bundle prompt engineering so that any host can reuse it. An MCP Prompt **does not call an LLM**; it returns messages and the host decides whether and how to send them to a model.

Why: five applications (Claims AI, Fraud AI, Adjuster AI, Underwriting AI, Customer Support AI) all need the same claim-analysis instructions. Hard-coding the prompt in each means duplicated maintenance and inconsistent behaviour. Exposing it once from an MCP server lets compatible applications retrieve the shared prompt.

```
Shared prompt
               Claims MCP Server
                       |
              Claim Analysis Prompt
                       |
          +------------+------------+
      Claims AI    Fraud AI    Adjuster AI
```

> **WATCH OUT** MCP makes the prompt available; it does not force every application to use it or guarantee identical LLM behaviour.

Enterprise examples: claims processing (analyse a claim and find missing information), underwriting (assess an application against guidelines), fraud investigation (summarise indicators for review), adjuster assistance (prepare a review checklist), treaty analysis (analyse a retrieved treaty section against a question).

### 4.4.2 MCP Prompt vs LangChain PromptTemplate

```
LangChain: prompt template inside your application
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    "Analyze this claim:\n{claim_details}\nIdentify coverage concerns.")
print(prompt.format(claim_details="CLM001: Windshield damage"))
```

| LangChain PromptTemplate | MCP Prompt |
|---|---|
| A prompt-formatting abstraction inside LangChain | A prompt exposed through the MCP protocol |
| Defined in application code | Defined and exposed by an MCP server |
| Formatted by the template implementation | Returned by the server through the MCP interface |
| No server needed | Retrievable by any compatible MCP client |
| Used in LCEL chains | Retrieved, then used by a host or framework |

They can work together: retrieve an MCP Prompt and feed the messages into a LangChain or LangGraph workflow.

### 4.4.3 Flow: prompts/list then prompts/get

```
Prompt retrieval
MCP Client -- prompts/list --> MCP Server -- prompt definitions --> MCP Client
MCP Client -- prompts/get (name + arguments) --> MCP Server
MCP Server substitutes the arguments -- formatted messages --> MCP Client -> Host -> (maybe) LLM
```

`prompts/list` returns names, descriptions and argument metadata. `prompts/get` returns the messages for one prompt. Retrieving the prompt and running the LLM are **two separate operations**.

### 4.4.4 Code (tested)

```
107_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Insurance Prompt Server")


@mcp.prompt()
def claim_analysis(claim_details: str, analysis_focus: str) -> str:
    """Generate a reusable prompt for insurance claim analysis."""
    return f"""You are an insurance claims analyst.
```

```
Analyze the following claim.

CLAIM DETAILS:
{claim_details}

ANALYSIS FOCUS:
{analysis_focus}

Instructions:
1. Summarize the relevant claim information.
2. Identify potential coverage concerns.
3. Identify missing information.
4. Highlight issues requiring human review.
5. Do not invent facts that are absent from the input.
6. Clearly distinguish facts from assumptions.

Return your response using these headings:
- Claim Summary
- Coverage Concerns
- Missing Information
- Items Requiring Human Review
"""


@mcp.prompt()
def fraud_review(claim_details: str, fraud_indicators: str) -> str:
    """Generate a reusable prompt for a fraud indicator review."""
    return f"""You are a fraud investigation assistant.

CLAIM DETAILS:
{claim_details}

REPORTED INDICATORS:
{fraud_indicators}

1. Summarize the supplied indicators.
2. Distinguish confirmed facts from potential concerns.
3. Identify missing information.
4. Recommend appropriate next investigative steps.
5. Do NOT declare fraud solely from an unverified indicator.
"""


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

**l07_client.py**

```
import asyncio
import sys
from pathlib import Path

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER = Path(__file__).with_name("l07_server.py")


async def main():
    params = StdioServerParameters(command=sys.executable, args=[str(SERVER)])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # prompts/list
            prompts = await session.list_prompts()
            print("========== AVAILABLE PROMPTS ==========")
```

```
            for p in prompts.prompts:
                print(f"\nName: {p.name}\nDescription: {p.description}")
                for arg in p.arguments or []:
                    print(f"  Argument: {arg.name} | required={arg.required}")

            # prompts/get  (arguments are always strings)
            result = await session.get_prompt(
                "claim_analysis",
                {"claim_details": "CLM001: Windshield damage, estimated loss 18500",
                 "analysis_focus": "Coverage and missing information"})
            print("\n========== prompts/get claim_analysis ==========")
            for message in result.messages:
                print(f"Role: {message.role}")
                print(message.content.text)

            result = await session.get_prompt(
                "fraud_review",
                {"claim_details": "CLM002: Collision claim",
                 "fraud_indicators": "Conflicting incident dates; documents not yet verified"})
            print("\n========== prompts/get fraud_review ==========")
            print(result.messages[0].content.text)


asyncio.run(main())
```

```
OUTPUT  python 107_client.py
========== AVAILABLE PROMPTS ==========

Name: claim_analysis
Description: Generate a reusable prompt for insurance claim analysis.
  Argument: claim_details | required=True
  Argument: analysis_focus | required=True

Name: fraud_review
Description: Generate a reusable prompt for a fraud indicator review.
  Argument: claim_details | required=True
  Argument: fraud_indicators | required=True

========== prompts/get claim_analysis ==========
Role: user
You are an insurance claims analyst.

Analyze the following claim.

CLAIM DETAILS:
CLM001: Windshield damage, estimated loss 18500

ANALYSIS FOCUS:
Coverage and missing information

Instructions:
1. Summarize the relevant claim information.
2. Identify potential coverage concerns.
3. Identify missing information.
4. Highlight issues requiring human review.
5. Do not invent facts that are absent from the input.
6. Clearly distinguish facts from assumptions.

Return your response using these headings:
- Claim Summary
- Coverage Concerns
- Missing Information
- Items Requiring Human Review


========== prompts/get fraud_review ==========
You are a fraud investigation assistant.
```

```
CLAIM DETAILS:
CLM002: Collision claim

REPORTED INDICATORS:
Conflicting incident dates; documents not yet verified

1. Summarize the supplied indicators.
2. Distinguish confirmed facts from potential concerns.
3. Identify missing information.
4. Recommend appropriate next investigative steps.
5. Do NOT declare fraud solely from an unverified indicator.
```

**TESTED** A prompt function that returns a `str` is delivered as a single message with role `user`. Prompt arguments always travel as **strings**.

`@mcp.prompt()` registers a prompt, exactly as `@mcp.tool()` registers a tool. The two arguments let a caller customise the prompt without duplicating the whole template: one application passes `analysis_focus="Coverage and missing information"`, another `"Claim completeness"`.

### 4.4.5 Modify and practise

- Add a third argument `response_format: str` and include `RESPONSE FORMAT: {response_format}` in the template; update the client to pass `"JSON"`. Both sides must change.
- The `fraud_review` prompt in the tested server is the practice task: it asks the model to summarise indicators, separate confirmed facts from concerns, identify missing information, recommend next investigative steps and avoid declaring fraud from an unverified indicator.

### 4.4.6 Prompts, Tools and Resources side by side

```
Three decorators, three interfaces
@mcp.tool()
def get_claim(claim_id: str) -> dict: ...

@mcp.resource("claims://{claim_id}")
def read_claim_resource(claim_id: str) -> str: ...

@mcp.prompt()
def claim_analysis(claim_details: str, analysis_focus: str) -> str: ...
```

- **Tool**: `get_claim("CLM001")`; the server executes the function and returns its result.
- **Resource**: `claims://CLM001`; the client reads identifiable content.
- **Prompt**: `claim_analysis(...)`; the client retrieves reusable instructions as messages.

### 4.4.7 Prompts in a LangGraph workflow

Instead of hard-coding the instructions inside a node, a node can retrieve the MCP Prompt, then pass the returned messages to the LLM:

```
Responsibilities
LangGraph node -> MCP Client -> retrieve claim_analysis prompt -> use messages -> LLM -> analysis
LangGraph: orchestrates the workflow.
MCP:       standard way to discover and retrieve the prompt.
LLM:       produces the analysis.
```

### 4.4.8 Prompt security

A prompt is not trustworthy just because it comes from an MCP server. Ask: who can modify it? who can retrieve it? could it contain instructions that conflict with application security policy? does the host validate it? are sensitive claim details included unnecessarily? is it version controlled? Prompts guide model behaviour; they do **not** replace access control or business-rule enforcement.

### 4.4.9 Interview questions on prompts

**Q. What is an MCP Prompt?**

A reusable prompt template exposed by an MCP server. A client discovers the available prompts and retrieves one with arguments; the host decides how to use the returned messages.

**Q. How is it different from a LangChain Prompt Template?**

A LangChain PromptTemplate is a formatting abstraction within LangChain. An MCP Prompt is exposed through the MCP protocol so compatible clients can discover and retrieve it. An application can retrieve an MCP Prompt and use it in a LangChain workflow.

**Q. Does an MCP Prompt call the LLM?**

Not by itself. It supplies instructions or messages; the host decides whether and how to send them to a model.

**Q. What do prompts/list and prompts/get do?**

`prompts/list` discovers available prompts and their metadata. `prompts/get` retrieves a selected prompt, optionally with arguments.

**Q. Can several AI applications reuse the same MCP Prompt?**

Yes, subject to access control and compatibility.

**Q. When would you use a Prompt instead of a Tool?**

A Prompt supplies reusable instructions or a message template. A Tool is for invoking a capability such as querying a database, calculating a reserve or verifying coverage.

## 4.5 Sampling

**Sampling** lets an MCP server request an LLM completion **from the host**. This enables agentic patterns where the server drives a reasoning loop without needing its own model API key.

- The server sends `sampling/createMessage` to the client.
- The client shows the request to the user for approval (human in the loop).
- The host calls the LLM and returns the result to the server.
- Useful for tool chains, agents-as-servers and structured data extraction.

> **TIP** Sampling is opt-in and powerful. The host can restrict or disable it through the capabilities exchanged at initialisation.

### 4.6 Roots

**Roots** are URIs the client exposes to tell the server the boundaries of its interest, for example a workspace folder `file:///home/user/project`. Servers use roots to scope operations and avoid touching unintended files.

### 4.7 Elicitation

**Elicitation** (2025 addition) lets a server ask the **end user** for extra input during a tool call. The server sends `elicitation/create`; the host shows a UI prompt and returns the answer. Use it when a tool needs information that was not available at call time, for example to confirm a destructive action or supply a missing parameter. It is gated by the client declaring elicitation support during initialisation.

### 4.8 Choosing between Tool, Resource and Prompt

| Primitive | Main question it answers | Example |
|---|---|---|
| Tool | What can the application **do**? | `calculate_reserve()` |
| Resource | What information can the application **read**? | `claims://CLM001` |
| Prompt | What reusable **instructions** can the application use? | `claim_analysis` |

The distinction is about the MCP **interface**, not the Python implementation: a tool can internally read data, and a resource handler can execute code. The interface tells the client how to use the capability. Section 12 combines all three in one workflow.

## 5. Building an MCP Server

### 5.1 Setup

The official Python SDK is the `mcp` package (Python 3.10 or newer). Always work in a virtual environment.

```
Windows
python -m venv venv
venv\Scripts\activate
python -m pip install mcp
python -m pip show mcp
```

```
Linux / Azure VM
python3 -m venv venv
source venv/bin/activate
python -m pip install mcp
```

```
Optional extras
pip install "mcp[cli]"      # adds the `mcp` command (mcp dev server.py) and HTTP extras
```

**VERSION NOTE** Everything in this document was run on `mcp` **1.30.0**. SDK APIs evolve (the study chat also referred to a newer generation with `MCPServer` and a URL-based `Client`). If an import or signature fails, check `pip show mcp` and the matching documentation rather than mixing examples from different SDK generations.

## 5.2 The minimal server

```
Smallest useful MCP server (FastMCP)
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("my-server")            # 1. create the server

@mcp.tool()                           # 2. expose a function as a tool
def ping(message: str) -> str:
    """Echo a message back."""
    return message

if __name__ == "__main__":
    mcp.run(transport="stdio")        # 3. run it (stdio here)
```

`FastMCP("name")` creates the server instance. The name identifies the server in metadata; it does **not** open an HTTP port. The decorator registers a function, the SDK builds the schema from the type hints and docstring, and `run()` starts the chosen transport.

## 5.3 Design the tools before writing code

An AI claims assistant needs three operations. Instead of implementing the business logic inside the AI application we expose them through an MCP server.

```
What we are building
USER -> Claims AI Assistant -> MCP Client --(MCP protocol)--> Claims MCP Server
                                                +-- get_claim
                                                +-- get_policy
                                                +-- calculate_reserve
                                                        |
                                                Business logic -> Claims / Policy data
```

| Tool | Input | Output (example) |
|---|---|---|
| `get_claim` | `claim_id` | `claim_id`, `policy_id`, `status`, `loss_type`, `estimated_amount` |
| `get_policy` | `policy_id` | `policy_id`, `status`, `coverage_limit`, `deductible` |
| `calculate_reserve` | `claim_id`, `severity_buffer` | `estimated_amount`, `severity_buffer`, `recommended_reserve` |

The reserve formula used here (educational only): recommended reserve = estimated loss x (1 + severity buffer). With a loss of 18,500 and a 10% buffer: 18,500 x 1.10 = **20,350**.

> **WATCH OUT** This is not a production insurance reserving rule. A real recommendation must reflect company rules, coverage, limits, prior payments and more.

## 5.4 The full server (tested)

```
105_server.py
import sys
from mcp.server.fastmcp import FastMCP

# 1. Initialise the MCP server
mcp = FastMCP("Claims MCP Server")

# 2. Simulated database
CLAIMS = {
    "CLM001": {"claim_id": "CLM001", "policy_id": "POL1001", "status": "OPEN",
```

```
                "loss_type": "Windshield Damage", "estimated_amount": 18500.0},
    "CLM002": {"claim_id": "CLM002", "policy_id": "POL1002", "status": "OPEN",
                "loss_type": "Collision", "estimated_amount": 75000.0},
    "CLM003": {"claim_id": "CLM003", "policy_id": "POL1003", "status": "CLOSED",
                "loss_type": "Vehicle Theft", "estimated_amount": 250000.0},
}

POLICIES = {
    "POL1001": {"policy_id": "POL1001", "status": "ACTIVE",
                "coverage_limit": 500000.0, "deductible": 1000.0},
    "POL1002": {"policy_id": "POL1002", "status": "ACTIVE",
                "coverage_limit": 300000.0, "deductible": 2500.0},
    "POL1003": {"policy_id": "POL1003", "status": "LAPSED",
                "coverage_limit": 250000.0, "deductible": 5000.0},
}


# 3. Tools ------------------------------------------------------------------
@mcp.tool()
def get_claim(claim_id: str) -> dict:
    """Retrieve claim details using a claim ID."""
    print(f"[SERVER] get_claim({claim_id})", file=sys.stderr)
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return {"error": "CLAIM_NOT_FOUND", "message": f"Claim {claim_id} does not exist."}
    return claim


@mcp.tool()
def get_policy(policy_id: str) -> dict:
    """Retrieve policy details using a policy ID."""
    policy = POLICIES.get(policy_id)
    if policy is None:
        return {"error": "POLICY_NOT_FOUND", "message": f"Policy {policy_id} does not exist."}
    return policy


@mcp.tool()
def calculate_reserve(claim_id: str, severity_buffer: float = 0.10) -> dict:
    """Calculate a simplified recommended reserve: estimated loss x (1 + buffer)."""
    if not 0 <= severity_buffer <= 1:
        return {"error": "INVALID_SEVERITY_BUFFER",
                "message": "Severity buffer must be between 0 and 1."}
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return {"error": "CLAIM_NOT_FOUND", "message": f"Claim {claim_id} does not exist."}
    estimated = claim["estimated_amount"]
    return {"claim_id": claim_id, "estimated_amount": estimated,
            "severity_buffer": severity_buffer,
            "recommended_reserve": round(estimated * (1 + severity_buffer), 2)}


@mcp.tool()
def get_claim_policy_link(claim_id: str) -> dict:
    """Retrieve the policy ID and policy status associated with a claim."""
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return {"error": "CLAIM_NOT_FOUND", "message": f"Claim {claim_id} does not exist."}
    policy = POLICIES.get(claim["policy_id"])
    if policy is None:
        return {"error": "POLICY_NOT_FOUND",
                "message": f"Policy {claim['policy_id']} does not exist."}
    return {"claim_id": claim_id, "policy_id": policy["policy_id"],
            "policy_status": policy["status"],
            "coverage_limit": policy["coverage_limit"]}


if __name__ == "__main__":
```

```
    mcp.run(transport="stdio")
```

### Understanding the server

**A. Initialise.** `FastMCP("Claims MCP Server")` creates the server. No port is opened.

**B. Register tools.** The decorator registers the function. The SDK uses the function name, docstring and type annotations to build the tool metadata and input schema (`claim_id: str` means a required string). Good descriptions matter because the LLM chooses tools from them:

```
Descriptions drive tool choice
get_claim          -> Retrieve claim details.
get_policy         -> Retrieve policy details.
calculate_reserve  -> Calculate a simplified reserve.
```

**C. Validate inputs.** `if not 0 <= severity_buffer <= 1` rejects a buffer outside 0 to 100%. Remember the difference: a **schema** describes the expected input *format*; **business validation** decides whether a value is acceptable for the operation. Validate at the business boundary; do not rely on the LLM to supply valid arguments.

**D. Return structured data.** The function returns a dict. The SDK converts it into an MCP tool result. The client must consume the **result object**, not assume it receives the original Python dict.

### Tool-design notes from the tested server

- Missing records return `{"error": "CLAIM_NOT_FOUND", ...}` as data. Remember: this arrives with `isError=False` (Section 4.3).
- `get_claim_policy_link` combines two data sources. In an enterprise system the same pattern would call a claims API and a policy API instead of two dictionaries.
- Diagnostics go to **stderr** (`print(..., file=sys.stderr)`) because the server uses stdio.

## 5.5 Inspect and debug with MCP Inspector

```
Interactive browser-based debugger
npx @modelcontextprotocol/inspector python 105_server.py

# or, with the mcp[cli] extra
mcp dev 105_server.py
```

The Inspector lists tools, resources and prompts and lets you call them by hand. It is the quickest way to see exactly what a server advertises.

## 5.6 A second example: a weather server

> **NOTE** This one was carried over from the first version of the notes. It needs internet access and was not executed here; treat it as an illustration of a tool that calls an external HTTP service.

```
weather_server.py (FastMCP version, not executed)
import httpx
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("weather-server")

@mcp.resource("weather://supported-cities")
def supported_cities() -> str:
```

```
    """List of supported city names."""
    return '["London", "New York", "Tokyo", "Sydney", "Dubai"]'

@mcp.tool()
async def get_weather(city: str) -> str:
    """Get current weather for a city. Use when the user asks about weather."""
    async with httpx.AsyncClient() as client:
        resp = await client.get(f"https://wttr.in/{city}?format=3")
        return resp.text.strip()

if __name__ == "__main__":
    mcp.run()
```

## 5.7 The low-level Server API

`FastMCP` is the high-level API used throughout this document. The SDK also has a low-level `Server` class for full control over request handling (custom list/call handlers, custom content types):

```
Low-level style (reference, not executed)
import asyncio
import mcp.types as types
from mcp.server import Server
from mcp.server.stdio import stdio_server

server = Server("my-server")

@server.list_tools()
async def list_tools() -> list[types.Tool]:
    return [types.Tool(name="ping", description="Echo",
                       inputSchema={"type": "object",
                                    "properties": {"message": {"type": "string"}},
                                    "required": ["message"]})]

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[types.TextContent]:
    if name == "ping":
        return [types.TextContent(type="text", text=arguments["message"])]
    raise ValueError(f"Unknown tool: {name}")

async def main():
    async with stdio_server() as (read, write):
        await server.run(read, write, server.create_initialization_options())

asyncio.run(main())
```

## 5.8 Trace one tool call through the server

`await session.call_tool("calculate_reserve", {"claim_id": "CLM001", "severity_buffer": 0.10})`

1. **Client prepares the call**: tool `calculate_reserve`, arguments `claim_id=CLM001`, `severity_buffer=0.10`.
2. **MCP client sends** a `tools/call` message (name plus arguments).
3. **Server dispatches** it to the registered `calculate_reserve` function.
4. **The function reads the claim**: `CLAIMS.get("CLM001")` finds the 18,500 record.
5. **The function calculates**: 18,500 x (1 + 0.10) = 20,350.
6. **The server returns** the value converted into an MCP tool result.
7. **The client receives it** and can pass it to an LLM, log it or use it in a workflow.

The server never needs to know which LLM will consume the result; it simply exposes a capability through the protocol.

## 5.9 Mapping the pattern to enterprise systems

Replace the dictionaries with real systems and the structure stays identical:

```
Policy coverage server in front of Guidewire
LLM -> MCP Client -> Policy MCP Server
                     +-- gw_search_policy ---+
                     +-- gw_get_coverage ----+--> Guidewire
```

```
Adjuster server in front of PostgreSQL
LLM -> MCP Client -> Adjuster MCP Server
                     +-- reserve recommendation
                     +-- settlement recommendation
                     +-- repair / replacement
                     +-- fraud information
                     +-- adjuster findings   --> PostgreSQL
```

A tool body can be a database query, a REST call or a Guidewire API call:

```
Possible tool bodies
@mcp.tool()
def get_claim(claim_id: str):
    response = requests.get(f"{CLAIMS_API}/{claim_id}")     # REST
    return response.json()

@mcp.tool()
def get_claim_db(claim_id: str):
    return db.execute("SELECT * FROM claims WHERE claim_id = %s", (claim_id,))  # SQL
```

For your Settlement Recommendation flow you might expose `get_policy_details`, `get_loss_estimation`, `get_repair_replacement_decision`, `get_fraud_snapshot`, `get_adjuster_findings` and `get_stp_score`. The LLM never sees PostgreSQL; it sees capabilities. The server handles the Tool, SQL, PostgreSQL, normalisation and MCP result chain.

## 5.10 Does the client execute the Python function?

No. The client never imports `get_claim`; it sends a request across the process boundary.

```
Separation
CLIENT PROCESS -- MCP request --> SERVER PROCESS -- registered tool --> get_claim()
```

Because of this the server and client can be maintained independently, and several compatible applications (Claims AI, Fraud AI, Adjuster AI, Underwriting AI) can consume the same server, subject to authentication, authorisation and deployment configuration.

## 5.11 Production design: what must change before deployment

- **Replace in-memory dictionaries** with repositories or services (`claims_repository.get_by_id(claim_id)`), separating the tool interface from database access.
- **Validate at the business boundary**: identifiers, numeric ranges, required fields and the permissions of the caller.
- **Protect sensitive information**: return only the fields the tool is authorised to disclose and the workflow needs.
- **Handle errors deliberately**: never leak SQL errors, internal URLs, credentials or stack traces to the model.

- **Add observability**: tool name, duration, success or failure, correlation id, sanitised error category. Avoid logging sensitive claim data.
- **Protect write operations**: reading a claim and creating a reserve have different risk. For `create_reserve` consider authorisation, limits, duplicate-request protection, audit trails and approvals.

> **WATCH OUT** An MCP tool is an interface to a capability, not a guarantee that invoking the capability is safe.

## 5.12 Interview questions on building servers

### Q. How do you expose a Python function as an MCP tool?

With the Python MCP SDK I register it using the `@mcp.tool()` decorator. The SDK exposes its metadata and input schema and handles invocation through the MCP protocol.

### Q. Why separate tool definitions from business logic?

It makes the code easier to test, maintain and replace. The tool acts as the interface while repositories or services handle data access and business operations.

### Q. What happens when an MCP client calls a tool?

The client sends a tool-call request through the MCP protocol; the server dispatches it to the registered tool, executes the business logic and returns the result in an MCP tool-result representation.

### Q. Can multiple AI applications use the same MCP server?

Yes, provided the server is reachable and access policies allow it.

### Q. Is an MCP server just a collection of Python functions?

No. Functions implement the capabilities, but the server also provides a protocol interface through which compatible clients discover and invoke them.

### Q. How would you connect an MCP tool to PostgreSQL?

The tool is an interface to a service or repository that runs parameterised SQL. I validate inputs, enforce authorisation, normalise the result and return only the required data.

### Q. How would you connect an MCP tool to Guidewire?

The tool calls the appropriate Guidewire API or an existing internal service, validates inputs, applies authorisation, normalises the response and returns a structured result.

# 6. Building an MCP Client

## 6.1 What a client does

A client connects to a server, negotiates capabilities, then calls tools or reads resources on behalf of the host. With the SDK the steps are: describe how to reach the server, open the transport, create a `ClientSession`, call `initialize()`, then use the session.

## 6.2 The full client for the claims server (tested)

**105_client.py**

```
import asyncio
import sys
from pathlib import Path

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER = Path(__file__).with_name("l05_server.py")


def show(title, result):
    print(f"\n--- {title} (isError={result.isError}) ---")
    for item in result.content:
        if hasattr(item, "text"):
            print(item.text)


async def main():
    params = StdioServerParameters(command=sys.executable, args=[str(SERVER)])

    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # Discovery: tools/list
            tools = await session.list_tools()
            print("========== AVAILABLE TOOLS ==========")
            for tool in tools.tools:
                print(f"- {tool.name}: {tool.description}")
            print("\nSchema of calculate_reserve:")
            print(tools.tools[2].inputSchema)

            # Invocation: tools/call
            show("get_claim", await session.call_tool(
                "get_claim", {"claim_id": "CLM001"}))
            show("get_policy", await session.call_tool(
                "get_policy", {"policy_id": "POL1001"}))
            show("calculate_reserve (valid)", await session.call_tool(
                "calculate_reserve", {"claim_id": "CLM001", "severity_buffer": 0.10}))
            show("calculate_reserve (buffer = 2.0, business error)", await session.call_tool(
                "calculate_reserve", {"claim_id": "CLM001", "severity_buffer": 2.0}))
            show("get_claim_policy_link", await session.call_tool(
                "get_claim_policy_link", {"claim_id": "CLM001"}))
            show("get_claim (unknown id)", await session.call_tool(
                "get_claim", {"claim_id": "CLM999"}))

            # Wrong argument type: rejected by schema validation
            show("calculate_reserve (bad type)", await session.call_tool(
                "calculate_reserve", {"claim_id": "CLM001", "severity_buffer": "lots"}))

            # Unknown tool
            show("unknown tool", await session.call_tool("delete_all_claims", {}))
```

```
asyncio.run(main())
```

Run it. The client launches the server itself over stdio:

```
Run
python l05_client.py
```

```
OUTPUT  python l05_client.py
========== AVAILABLE TOOLS ==========
- get_claim: Retrieve claim details using a claim ID.
- get_policy: Retrieve policy details using a policy ID.
- calculate_reserve: Calculate a simplified recommended reserve: estimated loss x (1 + buffer).
- get_claim_policy_link: Retrieve the policy ID and policy status associated with a claim.

Schema of calculate_reserve:
{'properties': {'claim_id': {'title': 'Claim Id', 'type': 'string'}, 'severity_buffer': {'default': 0.1, 'ti
    tle': 'Severity Buffer', 'type': 'number'}}, 'required': ['claim_id'], 'title': 'calculate_reserveArgume
    nts', 'type': 'object'}

--- get_claim (isError=False) ---
{
  "claim_id": "CLM001",
  "policy_id": "POL1001",
  "status": "OPEN",
  "loss_type": "Windshield Damage",
  "estimated_amount": 18500.0
}

--- get_policy (isError=False) ---
{
  "policy_id": "POL1001",
  "status": "ACTIVE",
  "coverage_limit": 500000.0,
  "deductible": 1000.0
}

--- calculate_reserve (valid) (isError=False) ---
{
  "claim_id": "CLM001",
  "estimated_amount": 18500.0,
  "severity_buffer": 0.1,
  "recommended_reserve": 20350.0
}

--- calculate_reserve (buffer = 2.0, business error) (isError=False) ---
{
  "error": "INVALID_SEVERITY_BUFFER",
  "message": "Severity buffer must be between 0 and 1."
}

--- get_claim_policy_link (isError=False) ---
{
  "claim_id": "CLM001",
  "policy_id": "POL1001",
  "policy_status": "ACTIVE",
  "coverage_limit": 500000.0
}

--- get_claim (unknown id) (isError=False) ---
{
  "error": "CLAIM_NOT_FOUND",
  "message": "Claim CLM999 does not exist."
}
```

```
--- calculate_reserve (bad type) (isError=True) ---
Error executing tool calculate_reserve: 1 validation error for calculate_reserveArguments
severity_buffer
  Input should be a valid number, unable to parse string as a number [type=float_parsing, input_value='lots'
    , input_type=str]
    For further information visit https://errors.pydantic.dev/2.14/v/float_parsing

--- unknown tool (isError=True) ---
Unknown tool: delete_all_claims
```

**TESTED** The last three blocks show the failure kinds from Section 4.3 in the real run: the business error (buffer 2.0) is `isError=False`; the wrong argument type and the unknown tool are `isError=True`. The server's stderr logging is hidden here because the output above shows only stdout.

### What the client does, step by step

```
Execution flow
client.py starts -> client launches server.py -> MCP session initialises
 -> client requests tools (tools/list) -> server returns 4 tool definitions
 -> call get_claim -> call get_policy -> call calculate_reserve
 -> client receives and prints the results
```

The client does not import `get_claim()`; it communicates through the MCP interface. The discovery/invocation split is the same as in Section 7: `list_tools()` to discover, `call_tool()` to invoke.

## 6.3 Capability discovery pattern

Always check what the server declared before calling optional features. Calling an unsupported method returns a JSON-RPC error.

```
Check capabilities safely
init = await session.initialize()          # returns the server's InitializeResult
caps = init.capabilities

if caps.tools:
    tools = await session.list_tools()
if caps.resources:
    resources = await session.list_resources()
    templates = await session.list_resource_templates()
if caps.prompts:
    prompts = await session.list_prompts()
```

## 6.4 Reading results correctly

| Field on the result | Meaning |
|---|---|
| `result.content` | List of content blocks (text, image, embedded resource). Dicts returned by simple tools arrive here as JSON text. |
| `result.structuredContent` | Typed structured output, when the tool declares an output schema (it was `None` for our plain `-> dict` tools). |
| `result.isError` | `True` for tool-level failures (validation, unknown tool, exceptions raised in the tool). |

```
Defensive parsing helper
import json

def parse_tool_result(result) -> dict:
    if result.isError:
        raise RuntimeError(f"MCP tool failed: {result.content[0].text}")
    data = json.loads(result.content[0].text)
    if "error" in data:                    # business error returned as data
        raise RuntimeError(f"Business error: {data}")
    return data
```

## 6.5 Direct call vs session.call_tool

```
Two very different things
get_claim("CLM001")                                    # direct Python call, same process
await session.call_tool("get_claim", {"claim_id": "CLM001"})   # MCP call across a process boundary
```

The first is a plain function call. The second uses the MCP client SDK to invoke a tool exposed by an MCP server: it builds a `tools/call` request, sends it over the transport, the server executes the registered function, and the SDK parses the result into a Python result object.

## 6.6 Practice tasks

- Add a tool `get_policy` returning `{"policy_id": "POL1001", "coverage_limit": 500000, "status": "ACTIVE"}`; call it with `await session.call_tool("get_policy", {"policy_id": "POL1001"})`.
- Call it with an invalid policy id and decide how the server should represent that business error (data vs `isError`).
- Write down the difference between the two calls in 6.5.

# 7. Protocol Messages Deep-Dive

## 7.1 The problem: how does a client tell a server to run a function?

A Claims MCP server exposes this Python function:

```
The function
def get_claim(claim_id: str) -> dict:
    return {"claim_id": claim_id, "status": "OPEN"}
```

In one Python process you just call `get_claim("CLM001")`. But an MCP client may live in another process, or on another machine; it cannot call the function directly. It must send a structured request that says: **which operation** it wants, **which tool**, **what arguments**, **which request the reply belongs to**, and it must be told whether the operation **succeeded or failed**. MCP uses **JSON-RPC 2.0** messages for this.

## 7.2 What is JSON-RPC 2.0

JSON-RPC is a protocol for making requests and receiving responses using JSON. Compare it with REST:

| REST | MCP (JSON-RPC) |
|---|---|
| Client sends an HTTP request; server returns an HTTP response | Client sends a JSON-RPC message; server returns a JSON-RPC response |
| The API defines what `POST /claims/search` means | The protocol defines operations such as `tools/call`; the tool itself is part of the MCP capability model |

| REST | MCP (JSON-RPC) |
|---|---|
| Transport and protocol are tied together | Protocol and transport are separate concepts |

```
A tools/call request
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "get_claim",
    "arguments": { "claim_id": "CLM001" }
  },
  "id": 1
}
```

| Field | Meaning |
|---|---|
| `jsonrpc` | Identifies the JSON-RPC protocol version (`"2.0"`) |
| `method` | The operation being requested (`tools/call`) |
| `params` | The operation's parameters (here: which tool and which arguments) |
| `id` | Identifies the request so its response can be matched to it |

> **KEY IDEA** `tools/call` is the **MCP operation**. `get_claim` is the **specific tool** being invoked.

Read the request almost as English: method = `tools/call`; tool = `get_claim`; arguments: `claim_id = CLM001`. This mirrors what you already know from LangGraph, where a tool call has `name` and `args`; MCP does the same thing at a protocol level.

### Why the id exists

Several requests can be in flight, and responses can return asynchronously:

```
Correlation
Request 1 -> get_claim(CLM123)    Response id 1 -> claim response
Request 2 -> get_policy(POL456)   Response id 2 -> policy response
Request 3 -> get_claim(CLM999)    Response id 3 -> claim response
```

The `id` lets the client correlate each response with its request. This is a common RPC pattern.

## 7.3 The three message types

| Type | Meaning | Has ? | Gets a response? |
|---|---|---|---|
| Request | "Perform an operation" | Yes | Yes |
| Response | "Here is the result" (or an error) | Yes (same as the request) | No |
| Notification | Tell the other side something happened | No | No |

```
Response and error response
{ "jsonrpc": "2.0", "result": { "claim_id": "CLM123", "status": "OPEN" }, "id": 1 }

{ "jsonrpc": "2.0", "error": { "code": -32602, "message": "Invalid claim ID" }, "id": 1 }
```

A success has `result`; a failure has `error`. Your application never needs to invent its own RPC message structure.

```
A notification (historical MCP initialization acknowledgement)
{ "jsonrpc": "2.0", "method": "notifications/initialized" }
```

**VERSION NOTE** The notification above illustrates the JSON-RPC shape and the historical MCP initialisation sequence. It is not a claim that every current MCP revision starts with the same handshake.

### Standard JSON-RPC error codes

| Code | Name | Meaning |
|---|---|---|
| -32700 | Parse error | Invalid JSON was received |
| -32600 | Invalid Request | The JSON is not a valid request object |
| -32601 | Method not found | The method does not exist |
| -32602 | Invalid params | Invalid method parameters |
| -32603 | Internal error | Internal JSON-RPC error |

### The MCP initialisation handshake (2025 model)

Connection setup is itself a small JSON-RPC conversation (illustrative shapes):

```
1) client -> server: initialize request
{ "jsonrpc": "2.0", "id": 1, "method": "initialize",
  "params": { "protocolVersion": "2025-03-26",
              "capabilities": { "sampling": {} },
              "clientInfo": { "name": "claims-ai", "version": "1.0" } } }
```

```
2) server -> client: result with the server's capabilities
{ "jsonrpc": "2.0", "id": 1,
  "result": { "protocolVersion": "2025-03-26",
              "capabilities": { "tools": {}, "resources": {}, "prompts": {} },
              "serverInfo": { "name": "Claims MCP Server", "version": "1.0" } } }
```

```
3) client -> server: notification (no id, no reply)
{ "jsonrpc": "2.0", "method": "notifications/initialized" }
```

**TESTED** The HTTP server log in Section 1.8 shows exactly this: initialize answered with 200, the `initialized` notification answered with **202 Accepted** (nothing to return), then `tools/list` and `tools/call` with 200.

## 7.4 A tiny JSON-RPC dispatcher in Python (tested)

Before using the SDK, it helps to see what a server does internally. This is an **educational** JSON-RPC-style dispatcher, not a full MCP server: it has no initialisation, no capability exchange, no transport handling and not the real MCP tool-result format.

```
l03_jsonrpc_demo.py
import json

# 1. Business logic ------------------------------------------------------
CLAIMS = {
    "CLM001": {"claim_id": "CLM001", "policy_id": "POL1001",
               "status": "OPEN", "estimated_amount": 18500},
    "CLM002": {"claim_id": "CLM002", "policy_id": "POL1002",
               "status": "CLOSED", "estimated_amount": 75000},
}
```

```
def get_claim(claim_id: str) -> dict:
    claim = CLAIMS.get(claim_id)
    if claim is None:
        raise ValueError(f"Claim {claim_id} not found")
    return claim


# 2. Tool registry: name (from the request) -> function (we registered) ------
TOOLS = {"get_claim": get_claim}


# 3. A tiny JSON-RPC style dispatcher (educational, NOT a full MCP server) ---
def handle_request(raw_request: str) -> str:
    request = json.loads(raw_request)          # JSON text -> dict
    request_id = request.get("id")
    method = request.get("method")
    params = request.get("params", {})
    print(f"[SERVER] Received method: {method}")

    try:
        if method != "tools/call":
            raise LookupError(f"Unknown method: {method}")

        tool = TOOLS.get(params.get("name"))
        if tool is None:
            raise KeyError(f"Unknown tool: {params.get('name')}")

        result = tool(**params.get("arguments", {}))   # dict -> kwargs
        response = {"jsonrpc": "2.0", "result": result, "id": request_id}

    except (LookupError, KeyError) as exc:
        response = {"jsonrpc": "2.0", "id": request_id,
                    "error": {"code": -32601, "message": str(exc).strip("'\"")}}
    except Exception as exc:                           # business failure
        response = {"jsonrpc": "2.0", "id": request_id,
                    "error": {"code": -32000, "message": str(exc)}}

    return json.dumps(response, indent=2)              # dict -> JSON text


# 4. A tiny client ------------------------------------------------------------
def send(request: dict) -> None:
    raw = json.dumps(request, indent=2)
    print("\n[CLIENT] Sending request:")
    print(raw)
    response = handle_request(raw)
    print("\n[SERVER] Sending response:")
    print(response)


if __name__ == "__main__":
    send({"jsonrpc": "2.0", "method": "tools/call", "id": 1,
          "params": {"name": "get_claim", "arguments": {"claim_id": "CLM001"}}})
    send({"jsonrpc": "2.0", "method": "tools/call", "id": 2,      # unknown tool
          "params": {"name": "delete_all_claims", "arguments": {}}})
    send({"jsonrpc": "2.0", "method": "tools/call", "id": 3,      # unknown claim
          "params": {"name": "get_claim", "arguments": {"claim_id": "CLM999"}}})
```

**Run**

```
python 103_jsonrpc_demo.py
```

```
OUTPUT  python 103_jsonrpc_demo.py
[CLIENT] Sending request:
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 1,
  "params": {
    "name": "get_claim",
    "arguments": {
      "claim_id": "CLM001"
    }
  }
}
[SERVER] Received method: tools/call

[SERVER] Sending response:
{
  "jsonrpc": "2.0",
  "result": {
    "claim_id": "CLM001",
    "policy_id": "POL1001",
    "status": "OPEN",
    "estimated_amount": 18500
  },
  "id": 1
}

[CLIENT] Sending request:
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 2,
  "params": {
    "name": "delete_all_claims",
    "arguments": {}
  }
}
[SERVER] Received method: tools/call

[SERVER] Sending response:
{
  "jsonrpc": "2.0",
  "id": 2,
  "error": {
    "code": -32601,
    "message": "Unknown tool: delete_all_claims"
  }
}

[CLIENT] Sending request:
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 3,
  "params": {
    "name": "get_claim",
    "arguments": {
      "claim_id": "CLM999"
    }
  }
}
[SERVER] Received method: tools/call

[SERVER] Sending response:
{
  "jsonrpc": "2.0",
  "id": 3,
  "error": {
```

```
    "code": -32000,
    "message": "Claim CLM999 not found"
  }
}
```

### Trace the execution

1. **The client builds a request**: a Python `dict`. No tool has run yet.
2. **Serialise**: `json.dumps(request)` turns the dict into JSON text. `dict` is a Python data structure; JSON is a data-interchange format. `json.dumps` serialises, `json.loads` parses.
3. **The server parses it**: `request = json.loads(raw_request)` then extracts `method` (`tools/call`) and `params`.
4. **Identify the tool**: `TOOLS.get(params["name"])`. The registry maps a name supplied by the request to a function the server registered.
5. **Execute**: `tool(**arguments)`. `The operator unpacks` {"claim_id": "CLM001"} `into keyword arguments, so this equals` get_claim(claim_id="CLM001")`.
6. **Build the response**: `{"jsonrpc": "2.0", "result": result, "id": request_id}`. The response keeps the request `id` so the client can correlate it.

### Break it: an unknown tool, and a missing claim

Request 2 asks for `delete_all_claims`. `TOOLS.get("delete_all_claims")` returns `None`, so the dispatcher answers with an error (-32601) instead of executing anything. **A server must never blindly execute arbitrary function names from a client**; production also needs authentication, authorisation and argument validation.

Request 3 asks for claim `CLM999`: the business function raises `ValueError("Claim CLM999 not found")`, the dispatcher catches it and returns an error response (the demo uses -32000).

| Failure | Meaning |
|---|---|
| Unknown JSON-RPC method | The requested operation is not supported |
| Unknown tool | The tool is not registered |
| Invalid arguments | The request does not provide acceptable arguments |
| Claim not found | The business record does not exist |

> **VERSION NOTE** In a real MCP implementation, protocol errors and tool execution errors follow the MCP specification and SDK conventions. This demo uses JSON-RPC error responses for everything and does not reproduce every distinction. In `mcp` 1.30.0 an unknown tool or a schema-violating argument comes back to the client as a normal result with `isError=true` (Section 4.3).

## 7.5 What the SDK does when you call session.call_tool

```
From Python call to protocol and back
session.call_tool()
      |
MCP client SDK
      |  builds the JSON-RPC request
transport (stdio / HTTP)
      |
MCP server SDK
      |  routes by tool name
get_claim()
      |
tool result -> MCP client SDK -> Python result object
```

You might think `call_tool` simply calls `get_claim("CLM001")`; in reality the call crosses the client/server boundary. The SDK builds the request, transmits it, parses the response and exposes a result object. What you did **not** have to write: request serialisation, response serialisation, session handling, protocol-level tool discovery, transport-level code.

## 7.6 The tool lifecycle: tools/list, then tools/call

When an interviewer asks "what actually happens when an LLM uses an MCP tool?" you should be able to explain two phases.

```
Phase 1: discovery.  Phase 2: invocation
Phase 1  MCP Client -- tools/list --> MCP Server -- tools + schemas --> MCP Client
Phase 2  LLM chooses a tool -> MCP Client -- tools/call --> MCP Server -> Tool
         -> Business system -> Tool result -> LLM
```

### Phase 1: tools/list

Request:

```
tools/list
{ "jsonrpc": "2.0", "method": "tools/list", "params": {}, "id": 1 }
```

Response (conceptually):

```
Tools with schemas
{
  "jsonrpc": "2.0",
  "result": { "tools": [
    { "name": "get_claim",
      "description": "Get claim details.",
      "inputSchema": { "type": "object",
                       "properties": { "claim_id": { "type": "string" } },
                       "required": ["claim_id"] } },
    { "name": "get_policy",
      "description": "Get policy details.",
      "inputSchema": { "type": "object",
                       "properties": { "policy_id": { "type": "string" } },
                       "required": ["policy_id"] } }
  ] },
  "id": 1
}
```

Now the client knows what tools exist, what they are called, what they do and what arguments they accept. That is **capability discovery**. Today a server exposes `get_claim` and `get_policy`; tomorrow it adds `get_exposure`, `create_reserve`, `check_fraud`, `get_adjuster_findings`. The AI application does not need a hard-coded list; it can discover them.

### The LLM decision

User: "What is the status of claim CLM001?" The application knows it has `get_claim(claim_id)`. The LLM reasons: the user wants claim information; the available tool is `get_claim`; the required argument is `claim_id`; so `claim_id = "CLM001"`. It produces a tool-call decision:

```
The model's decision (not executed by the model)
{ "name": "get_claim", "arguments": { "claim_id": "CLM001" } }
```

**KEY IDEA** The LLM does **not** execute Python. It says "I want this capability invoked with these arguments". The MCP client talks to the server; the server executes the registered tool.

## Phase 2: tools/call

```
tools/call
{ "jsonrpc": "2.0", "method": "tools/call",
  "params": { "name": "get_claim", "arguments": { "claim_id": "CLM001" } },
  "id": 2 }
```

The server looks up `get_claim` among its registered tools and effectively performs `get_claim(claim_id="CLM001")`. The body can be the dictionary lookup in our demo, or a REST call, a SQL query or a Guidewire API call. The result is carried back:

```
Result (illustrative; the exact shape depends on SDK and protocol version)
{ "jsonrpc": "2.0",
  "result": { "content": [ { "type": "text",
                             "text": "{\"claim_id\":\"CLM001\",\"status\":\"OPEN\"}" } ] },
  "id": 2 }
```

Do not memorise this JSON as a universal literal; memorise the architecture: tool execution, MCP result, MCP client.

## The result goes back to the LLM

The application feeds the result into the LLM's context, and the LLM writes the answer ("Claim CLM001 is currently open, with an estimated loss of 18,500").

```
The complete tool lifecycle
USER -> HOST (LLM) -- tool decision --> MCP CLIENT -- tools/call --> MCP SERVER -> Tool
        -> Enterprise API -> Tool result -> MCP CLIENT -> LLM -> Final answer
```

## Compared with the LangGraph ToolNode loop

```
LangGraph
LLM -> AIMessage.tool_calls -> ToolNode -> Python function -> ToolMessage -> LLM
```

```
MCP
LLM -> tool decision -> MCP Client -> tools/call -> MCP Server -> MCP Tool
    -> Tool result -> MCP Client -> LLM
```

LangGraph's `ToolNode` is an application/framework abstraction. MCP's `tools/call` is part of the **interoperability protocol**. In LangGraph a tool call looks like `{"name": "calculator", "args": {"a": 10, "b": 20}, "id": "call_123"}`; MCP does the same thing at protocol level.

## Why schemas matter

For `create_reserve(claim_id: str, amount: float, reason: str)` the schema tells the client/LLM that `claim_id` is a string, `amount` a number and `reason` a string. Without a structured contract you would have "create a reserve somehow"; with one you get `{"claim_id": "CLM001", "amount": 25000, "reason": "Windshield replacement"}`, which is far more deterministic.

### An end-to-end answer for interviews

1. The AI host contains the LLM and the MCP client.
2. The MCP client discovers server capabilities, for example with `tools/list`.
3. The tool definitions include names, descriptions and input schemas.
4. Based on the user's request, the LLM decides a particular tool should be invoked.
5. The MCP client sends `tools/call` with the tool name and structured arguments.
6. The MCP server receives the request and executes the corresponding tool.
7. The tool may talk to databases, REST APIs, Guidewire or other systems.
8. The server returns the result through MCP.
9. The client gives the result back to the AI application/LLM.
10. The LLM uses the result to generate the final response.

## 7.7 Does MCP replace REST?

No. A REST API defines its own endpoints; MCP defines a protocol-level capability model (tools, resources, prompts) and a discovery mechanism. An MCP server often calls REST APIs internally. Also remember that an MCP server is a logical protocol server, not "a REST server" (Section 1.6).

## 7.8 Message reference tables

**Lifecycle**

| Method | Direction | Purpose |
|---|---|---|
| `initialize` | Client to Server | Open the connection; exchange versions and capabilities |
| `notifications/initialized` | Client to Server | Acknowledge that initialisation is complete |
| `ping` | Either way | Health check; the other side replies with an empty result |
| `notifications/cancelled` | Either way | Cancel an in-flight request by id |

**Tools**

| Method | Direction | Purpose |
|---|---|---|
| `tools/list` | Client to Server | Discover tools and their input schemas |
| `tools/call` | Client to Server | Invoke a tool by name with arguments |
| `notifications/tools/list_changed` | Server to Client | The tool list changed |

**Resources**

| Method | Direction | Purpose |
|---|---|---|
| `resources/list` | Client to Server | Enumerate static resources |
| `resources/templates/list` | Client to Server | Enumerate URI templates |
| `resources/read` | Client to Server | Read one resource by URI |
| `resources/subscribe` / `resources/unsubscribe` | Client to Server | Start or stop change notifications for a URI |
| `notifications/resources/list_changed` | Server to Client | The resource list changed |
| `notifications/resources/updated` | Server to Client | A subscribed resource changed |

**Prompts**

| Method | Direction | Purpose |
|---|---|---|
| `prompts/list` | Client to Server | Discover prompts and their arguments |
| `prompts/get` | Client to Server | Retrieve a prompt (messages), optionally with arguments |
| `notifications/prompts/list_changed` | Server to Client | The prompt list changed |

**Client features and utilities**

| Method | Direction | Purpose |
|---|---|---|
| `sampling/createMessage` | Server to Client | Ask the host for an LLM completion |
| `elicitation/create` | Server to Client | Ask the user for more input |
| `roots/list` | Server to Client | Ask for the roots the client exposes |
| `notifications/message` | Server to Client | Structured log message |
| `notifications/progress` | Either way | Progress for a long-running request |

### Operations vs SDK calls

| SDK call | Protocol operation |
|---|---|
| `session.list_tools()` | `tools/list` |
| `session.call_tool(...)` | `tools/call` |
| `session.list_resources()` | `resources/list` |
| `session.list_resource_templates()` | `resources/templates/list` |
| `session.read_resource(uri)` | `resources/read` |
| `session.list_prompts()` | `prompts/list` |
| `session.get_prompt(name, args)` | `prompts/get` |

## 7.9 Interview questions on the protocol

**Q. What is JSON-RPC 2.0 in MCP?**

It is the remote-procedure-call message protocol used by the relevant MCP protocol versions. It defines structured requests, responses and notifications, and MCP uses these for operations such as discovering and invoking tools.

**Q. What is the purpose of the JSON-RPC id field?**

It correlates a response with its request. A response uses the same id as the request. Notifications have no id and receive no response.

**Q. What is the difference between tools/list and tools/call?**

`tools/list` discovers the tools a server exposes and their metadata. `tools/call` invokes a particular tool with its arguments.

**Q. Does the LLM directly execute the MCP server's Python function?**

No. The LLM may decide a tool should be called and produce the arguments. The host's MCP client communicates with the server, and the server executes the registered tool.

### Q. Why use the SDK instead of implementing JSON-RPC manually?

The SDK handles protocol communication, session management, tool registration, transport handling and request/response processing. That reduces the risk of implementing the protocol incorrectly and lets you focus on business logic.

### Q. How does an MCP client discover tools?

It uses `tools/list`, which returns names, descriptions and input schemas. To run a tool it uses `tools/call` with the tool name and structured arguments; the server executes it and returns the result.

### Q. Is "MCP just REST"?

No. REST defines endpoints; MCP defines an AI-oriented capability model and discovery on top of a message protocol. MCP servers can call REST internally.

## 7.10 Practice

- Add a tool `get_policy` and call it with the MCP client; call it with an invalid id.
- In the dispatcher demo add a `tools/list` method that returns the registry names.
- Explain the difference between `get_claim("CLM001")` and `await session.call_tool("get_claim", {...})`.

# 8. Authentication and Security

## 8.1 Authentication vs authorisation

| Question | Concept | Example |
|---|---|---|
| **Who** is calling? | Authentication | A verified access token says "this is alice" |
| **What** may they do? | Authorisation | Alice may read claims, may update only her own claims |

An MCP server can be perfectly authenticated and still unsafe if it never checks authorisation. Two levels of authorisation matter in a claims system:

- **Tool-level (scopes)**: can this caller use `update_reserve` at all? (needs the `claims:write` scope)
- **Record-level**: can this caller touch **this** claim? (alice may update CLM001 because it is assigned to her; she may not update bob's CLM002 even though she holds `claims:write`)

## 8.2 Tool discovery does not grant permission

`tools/list` tells a client what exists; it does not mean the caller may run it. Permission is enforced when the tool is **called**, on the server. A well-behaved server may also filter what `tools/list` returns per user, but it must never rely on the model or the client to hide a dangerous tool. Exposing a tool is not the same as authorising its use.

## 8.3 OAuth 2.1 in MCP (HTTP transports)

For HTTP-based MCP servers the specification builds on OAuth so clients can authenticate without handling credentials directly.

- The server publishes discovery metadata under `/.well-known/` so a client can find the authorisation server.
- Authorization Code flow with **PKCE** is the recommended flow for user-facing clients; Client Credentials for machine-to-machine.

- The client sends the access token on every request: `Authorization: Bearer <token>`.
- The server **validates the token on every request** (signature, expiry, issuer, audience) and extracts the identity and **scopes**.
- Scopes are narrow permissions such as `claims:read` and `claims:write`. Request the minimum.

```
Authorisation server metadata (discovery document)
{
  "issuer": "https://api.example.com",
  "authorization_endpoint": "https://api.example.com/oauth/authorize",
  "token_endpoint": "https://api.example.com/oauth/token",
  "response_types_supported": ["code"],
  "code_challenge_methods_supported": ["S256"]
}
```

**VERSION NOTE** Later spec revisions separate the roles: the MCP server acts as an OAuth **resource server** (it validates tokens) and points clients to a separate authorisation server, rather than being the authorisation server itself.

## 8.4 A real token-protected MCP server (tested)

The SDK provides `TokenVerifier` and `AuthSettings`. Your verifier turns a bearer token into an `AccessToken` (identity plus scopes), or returns `None` to reject it. Inside a tool, `get_access_token()` returns the **verified** identity of the current request. Below, the demo token store stands in for real JWT or introspection verification.

```
l08_auth_server.py
"""MCP server over HTTP that REQUIRES a bearer token, with scope and record checks."""
from mcp.server.auth.middleware.auth_context import get_access_token
from mcp.server.auth.provider import AccessToken, TokenVerifier
from mcp.server.auth.settings import AuthSettings
from mcp.server.fastmcp import FastMCP

# Demo "token store". A real verifier checks a JWT signature, expiry, issuer and
# audience, or calls the identity provider's introspection endpoint.
TOKENS = {
    "token-alice": AccessToken(token="token-alice", client_id="alice",
                               scopes=["claims:read", "claims:write"]),
    "token-carol": AccessToken(token="token-carol", client_id="carol",
                               scopes=["claims:read"]),
}


class DemoTokenVerifier(TokenVerifier):
    async def verify_token(self, token: str) -> AccessToken | None:
        return TOKENS.get(token)          # None -> the SDK answers 401


mcp = FastMCP(
    "Secure Claims Server",
    token_verifier=DemoTokenVerifier(),
    auth=AuthSettings(
        issuer_url="https://auth.example.com",          # who issues the tokens
        resource_server_url="http://127.0.0.1:8000",    # this server
        required_scopes=["claims:read"],                # needed for ANY request
    ),
)

CLAIMS = {
    "CLM001": {"status": "OPEN", "assigned_to": "alice", "reserve": 20000},
    "CLM002": {"status": "OPEN", "assigned_to": "bob", "reserve": 50000},
}
```

```
def current_user() -> AccessToken:
    token = get_access_token()            # the VERIFIED identity of this request
    assert token is not None
    return token


@mcp.tool()
def get_claim(claim_id: str) -> dict:
    """Read a claim you are assigned to."""
    me = current_user()
    claim = CLAIMS.get(claim_id)
    if claim is None:
        raise ValueError("claim not found")
    if claim["assigned_to"] != me.client_id:                # record-level check
        raise PermissionError(f"{claim_id} is not assigned to {me.client_id}")
    return claim


@mcp.tool()
def update_reserve(claim_id: str, amount: float) -> dict:
    """Update the reserve of a claim you are assigned to (needs claims:write)."""
    me = current_user()
    if "claims:write" not in me.scopes:                     # tool-level check
        raise PermissionError("missing scope claims:write")
    claim = CLAIMS.get(claim_id)
    if claim is None or claim["assigned_to"] != me.client_id:
        raise PermissionError(f"{claim_id} is not assigned to {me.client_id}")
    claim["reserve"] = amount
    return claim


if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

**108_auth_client.py**

```
import asyncio

import httpx
from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

URL = "http://127.0.0.1:8000/mcp"


async def run(label, token, tool, args):
    headers = {"Authorization": f"Bearer {token}"} if token else {}
    try:
        async with streamablehttp_client(URL, headers=headers) as (read, write, _):
            async with ClientSession(read, write) as session:
                await session.initialize()
                result = await session.call_tool(tool, args)
                text = result.content[0].text.replace("\n", " ")
                status = "DENIED " if result.isError else "ALLOWED"
                print(f"{label:<40} -> {status} {text[:90]}")
    except BaseException as exc:                        # connection-level rejection
        root = exc.exceptions[0] if isinstance(exc, BaseExceptionGroup) else exc
        if isinstance(root, httpx.HTTPStatusError):
            print(f"{label:<40} -> HTTP {root.response.status_code} {root.response.reason_phrase}")
        else:
            print(f"{label:<40} -> {type(root).__name__}: {root}")


async def main():
    await run("alice reads her claim", "token-alice", "get_claim", {"claim_id": "CLM001"})
    await run("alice reads bob's claim", "token-alice", "get_claim", {"claim_id": "CLM002"})
```

```
    await run("alice updates her reserve", "token-alice", "update_reserve",
              {"claim_id": "CLM001", "amount": 22000})
    await run("carol (read-only) updates reserve", "token-carol", "update_reserve",
              {"claim_id": "CLM001", "amount": 1})
    await run("no token", None, "get_claim", {"claim_id": "CLM001"})
    await run("invalid token", "token-evil", "get_claim", {"claim_id": "CLM001"})


asyncio.run(main())
```

**Run**

```
# Terminal 1
python l08_auth_server.py
# Terminal 2
python l08_auth_client.py
```

**OUTPUT** real run against the token-protected server

```
alice reads her claim                    -> ALLOWED {   "status": "OPEN",   "assigned_to": "alice",   "reser
    ve": 20000 }
alice reads bob's claim                  -> DENIED  Error executing tool get_claim: CLM002 is not assigned t
    o alice
alice updates her reserve                -> ALLOWED {   "status": "OPEN",   "assigned_to": "alice",   "reser
    ve": 22000.0 }
carol (read-only) updates reserve        -> DENIED  Error executing tool update_reserve: missing scope claim
    s:write
no token                                 -> HTTP 401 Unauthorized
invalid token                            -> HTTP 401 Unauthorized
```

**TESTED** The two missing/invalid-token cases are rejected **before any MCP message is processed**, with a plain HTTP 401. The scope failure (carol, read-only) and the record failure (alice reading bob's claim) are raised inside the tool, so they arrive as `isError=True` results with the message "Error executing tool ...". Alice can read and update only her own claim.

**WATCH OUT** Never trust a user id that the **model** puts in a tool argument. Take identity from the verified token (`get_access_token()`), not from tool arguments.

## The same logic without a server (easy to experiment with)

A pure-Python version of the two authorisation levels, runnable anywhere:

**l08_authz_demo.py**

```
"""Authentication vs authorization, scopes, and record-level checks.

In a real MCP HTTP server the TOKEN is verified (signature, expiry, issuer,
audience) by your auth layer, which hands the tool a verified identity.
Here the verified identities are simulated by a dictionary.
"""

# --- what a verified token gives us (authentication) -----------------------
TOKENS = {
    "token-alice": {"user": "alice", "scopes": {"claims:read", "claims:write"}},
    "token-bob":   {"user": "bob",   "scopes": {"claims:read", "claims:write"}},
    "token-carol": {"user": "carol", "scopes": {"claims:read"}},
}

# --- business data, including which adjuster owns each claim ---------------
CLAIMS = {
    "CLM001": {"status": "OPEN", "assigned_to": "alice", "reserve": 20000},
    "CLM002": {"status": "OPEN", "assigned_to": "bob",   "reserve": 50000},
}
```

```
class AuthError(Exception):
    pass


def authenticate(token: str) -> dict:
    """Authentication: WHO is calling? (invalid/expired token -> 401)"""
    identity = TOKENS.get(token)
    if identity is None:
        raise AuthError("401 Unauthorized: invalid or expired token")
    return identity


def require_scope(identity: dict, scope: str):
    """Authorization level 1: tool-level permission via OAuth scope (403)."""
    if scope not in identity["scopes"]:
        raise AuthError(f"403 Forbidden: missing scope {scope!r}")


def require_assignment(identity: dict, claim_id: str):
    """Authorization level 2: record-level permission (403)."""
    claim = CLAIMS.get(claim_id)
    if claim is None:
        raise AuthError("404 Not Found")
    if claim["assigned_to"] != identity["user"]:
        raise AuthError(f"403 Forbidden: {claim_id} is not assigned to "
                        f"{identity['user']}")


# --- two "tools" ----------------------------------------------------------
def get_claim(token: str, claim_id: str) -> dict:
    who = authenticate(token)
    require_scope(who, "claims:read")
    require_assignment(who, claim_id)
    return CLAIMS[claim_id]


def update_reserve(token: str, claim_id: str, amount: float) -> dict:
    who = authenticate(token)
    require_scope(who, "claims:write")        # tool-specific scope
    require_assignment(who, claim_id)         # record-specific check
    CLAIMS[claim_id]["reserve"] = amount
    return CLAIMS[claim_id]


def attempt(label, fn, *args):
    try:
        print(f"{label:<46} -> ALLOWED  {fn(*args)}")
    except AuthError as exc:
        print(f"{label:<46} -> DENIED   {exc}")


if __name__ == "__main__":
    attempt("alice reads her claim (CLM001)", get_claim, "token-alice", "CLM001")
    attempt("alice reads bob's claim (CLM002)", get_claim, "token-alice", "CLM002")
    attempt("alice updates her claim", update_reserve, "token-alice", "CLM001", 22000)
    attempt("alice updates bob's claim", update_reserve, "token-alice", "CLM002", 1)
    attempt("carol (read-only) reads CLM001 (not hers)", get_claim, "token-carol", "CLM001")
    attempt("carol updates CLM001 (no write scope)", update_reserve, "token-carol", "CLM001", 1)
    attempt("unknown token", get_claim, "token-evil", "CLM001")
```

```
OUTPUT  python l08_authz_demo.py
alice reads her claim (CLM001)                -> ALLOWED  {'status': 'OPEN', 'assigned_to': 'alice', 'reser
    ve': 20000}
alice reads bob's claim (CLM002)              -> DENIED   403 Forbidden: CLM002 is not assigned to alice
alice updates her claim                       -> ALLOWED  {'status': 'OPEN', 'assigned_to': 'alice', 'reser
    ve': 22000}
```

```
alice updates bob's claim                    -> DENIED   403 Forbidden: CLM002 is not assigned to alice
carol (read-only) reads CLM001 (not hers)    -> DENIED   403 Forbidden: CLM001 is not assigned to carol
carol updates CLM001 (no write scope)        -> DENIED   403 Forbidden: missing scope 'claims:write'
unknown token                                -> DENIED   401 Unauthorized: invalid or expired token
```

HTTP status meaning: **401 Unauthorized** = I do not know who you are (missing or invalid token); **403 Forbidden** = I know who you are and you are not allowed.

## 8.5 HTTP vs stdio security

| | stdio | HTTP |
|---|---|---|
| Trust boundary | The OS process: the server inherits the privileges of the process that spawned it | The network: any reachable client can try to connect |
| Authentication | Usually none; the local user launched the process | Required: OAuth/bearer tokens, validated on every request |
| Transport protection | Not applicable (pipes) | TLS (HTTPS) is mandatory in production |
| Main risk | Running the server as root or with broad file access; secrets in the config's `env` | Token theft, missing authorisation, exposed endpoints |

**WATCH OUT** Never run a stdio MCP server as root. Give it the minimum OS permissions it needs.

## 8.6 Security best practices

- **Least privilege**: expose only what is needed; separate read tools from write tools; give an agent only the scopes it needs (a fraud agent that reads claims must not get payment capabilities).
- **Input validation**: validate tool arguments against the declared schema **and** against business rules.
- **Treat all content as untrusted**: claim descriptions, uploaded documents and retrieved records can contain instructions aimed at the model (prompt injection). Treat them as data, never as trusted system instructions. Do not let a tool description be built from user-controlled text.
- **TLS everywhere** for HTTP transports.
- **Token scoping**: narrow OAuth scopes; no admin scopes for a read-only server.
- **Audit logging**: log every tool call with caller identity and arguments (without secrets and unnecessary PII).
- **Rate limiting**: protect expensive tools from accidental or malicious loops.
- **Do not leak internals**: no SQL errors, stack traces, internal URLs or credentials in tool results.
- **Protect write tools**: duplicate protection (idempotency keys), limits, audit trail, human approval for high-impact actions.

## 8.7 What the guided Lesson 8 audit found (and how it is now closed)

The study chat ended Lesson 8 with a five-point checklist and an honest audit: concepts covered, but three items only partly demonstrated because the code had not been run. Here is the status now:

| Topic | Status in the transcript | Where it is covered and tested |
|---|---|---|
| Authentication vs authorisation | Covered | 8.1 |
| Tool discovery does not grant permission | Covered | 8.2 |
| OAuth tokens, scopes, server-side verification | Partial (demo tokens, nothing run) | 8.3 and 8.4: real token verifier, real 401 |

| Topic | Status in the transcript | Where it is covered and tested |
|---|---|---|
| Tool-specific and record-specific authorisation | Partial (described, not implemented) | 8.4 and the pure-Python demo: alice vs bob, carol read-only |
| HTTP vs stdio security | Partial | 8.5 |

Still not shown here: a complete OAuth 2.1 authorisation-code flow against a real identity provider, and JWT signature verification. In production use your identity provider's library for those and keep the verifier pattern from 8.4.

## 8.8 Lesson 8 checklist

- [ ] I can distinguish authentication from authorisation.
- [ ] I understand why tool discovery does not grant permission.
- [ ] I can explain OAuth tokens, scopes and server-side verification.
- [ ] I understand why tool-specific and record-specific checks are both needed.
- [ ] I can explain the security difference between HTTP and stdio.

## 8.9 Interview questions on security

**Q. What is the difference between authentication and authorisation in an MCP server?**

Authentication establishes who is calling, for example by validating a bearer token. Authorisation decides what that identity may do, at tool level with scopes and at record level with business rules such as claim assignment.

**Q. Does tools/list grant permission to call a tool?**

No. It only describes what exists. Permission must be enforced on the server when the tool is invoked.

**Q. How would you secure an HTTP-based MCP server?**

Use HTTPS, OAuth 2.1 with PKCE for user-facing clients, validate the bearer token on every request, use narrow scopes, validate arguments against schema and business rules, take identity from the verified token rather than from arguments, and add rate limiting and audit logging.

**Q. Why check both scope and record ownership?**

A scope only says the caller may use a kind of operation. Without a record-level check, any user with `claims:write` could modify anyone's claim.

**Q. What is the security model of stdio?**

The process boundary. The server inherits the privileges of the process that launched it and the client talks to it through pipes, so there is no network exposure, but the server must still run with minimal OS permissions and its configuration secrets must be protected.

# 9. Advanced Features

## 9.1 Pagination

List operations (`tools/list`, `resources/list`, `prompts/list`) use **cursor-based pagination** when a server has many items. The response may include a `nextCursor`; the client passes it back to get the next page, until no cursor is returned.

```
Pagination loop (standard pattern, not executed here)
tools, cursor = [], None
while True:
    page = await session.list_tools(cursor=cursor)
    tools.extend(page.tools)
    cursor = page.nextCursor
    if not cursor:
        break
```

## 9.2 Progress reporting and logging (tested)

Long-running tools can report progress and emit structured log messages. In FastMCP both come from the `Context` object injected into the tool. On the client you pass a `progress_callback` to `call_tool` and a `logging_callback` to the session.

```
adv_progress_server.py
import asyncio
from mcp.server.fastmcp import FastMCP, Context

mcp = FastMCP("Export Server")


@mcp.tool()
async def export_claims(rows: int, ctx: Context) -> str:
    """Export claims, reporting progress and log messages as it goes."""
    await ctx.info(f"Started processing {rows} rows")           # notifications/message
    for i in range(1, rows + 1):
        await asyncio.sleep(0.05)                               # pretend work
        await ctx.report_progress(i, rows, f"row {i}/{rows}")    # notifications/progress
    await ctx.info("Finished")
    return f"exported {rows} rows"


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

```
adv_progress_client.py
import asyncio
import sys
from pathlib import Path

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER = Path(__file__).with_name("adv_progress_server.py")


async def on_progress(progress: float, total: float | None, message: str | None):
    print(f"[PROGRESS] {progress}/{total} {message or ''}")


async def on_log(params):
```

```
    print(f"[LOG:{params.level}] {params.data}")


async def main():
    params = StdioServerParameters(command=sys.executable, args=[str(SERVER)])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write, logging_callback=on_log) as session:
            await session.initialize()
            result = await session.call_tool(
                "export_claims", {"rows": 3}, progress_callback=on_progress)
            print("[RESULT]", result.content[0].text)


asyncio.run(main())
```

```
OUTPUT  python adv_progress_client.py
[LOG:info] Started processing 3 rows
[PROGRESS] 1.0/3.0 row 1/3
[PROGRESS] 2.0/3.0 row 2/3
[PROGRESS] 3.0/3.0 row 3/3
[LOG:info] Finished
[RESULT] exported 3 rows
```

Underneath, progress is a `notifications/progress` notification and logs are `notifications/message`. Log levels: debug, info, notice, warning, error, critical, alert, emergency. The client identifies the call to report on by a progress token the SDK sets for you.

## 9.3 Cancellation

A client can cancel an in-flight request by sending `notifications/cancelled` with the id of the original request. The server should stop work if it can; long loops should check for cancellation. Remember that cancelling locally does not prove the server stopped, and a cancelled write may or may not have been applied (Section 10).

## 9.4 Change notifications and subscriptions

Servers can notify clients when their lists change (`notifications/tools/list_changed`, `notifications/resources/list_changed`, `notifications/prompts/list_changed`) and, for resources a client subscribed to, when the content changes (`notifications/resources/updated`). This keeps a host's cached capability list fresh without polling.

## 9.5 Batch requests

JSON-RPC 2.0 allows sending an array of requests in one message and receiving an array of responses. The 2025-03-26 MCP revision allowed batches on HTTP transports.

> **VERSION NOTE** Treat batching as something you should not rely on; I believe a later spec revision removed it. Verify against the revision you target.

## 9.6 Sampling, Roots and Elicitation in code

> **NOTE** These three client-side features need a host that implements them, so they were not executed here. They are shown so you recognise the shapes in interviews.

```
Server asking the host for an LLM completion (sampling) and the user for input (elicitation)
from pydantic import BaseModel
from mcp.server.fastmcp import FastMCP, Context
from mcp.types import SamplingMessage, TextContent

mcp = FastMCP("demo")

class Confirm(BaseModel):
    approve: bool

@mcp.tool()
async def risky_action(claim_id: str, ctx: Context) -> str:
    answer = await ctx.elicit(message=f"Really close claim {claim_id}?", schema=Confirm)
    if answer.action != "accept" or not answer.data.approve:
        return "cancelled by user"

    result = await ctx.session.create_message(            # sampling/createMessage
        messages=[SamplingMessage(role="user",
                  content=TextContent(type="text", text="Summarise the claim in one line."))],
        max_tokens=100)
    return result.content.text
```

# 10. Error Handling, Timeouts, Retries and Reliability

The question for this section: **what happens when an MCP tool fails while an AI agent is using it?** Imagine the claims agent needs to (1) retrieve a claim from a claims API, (2) retrieve its policy, (3) calculate a reserve recommendation, (4) return it to the adjuster. The API may be slow, the database may be briefly down, or a tool may receive invalid arguments. Without handling, one failure breaks the whole workflow.

## 10.1 The four things to handle

| Concern | Question it answers |
|---|---|
| Error handling | What should happen when the tool fails: return an error, use a fallback, or stop? |
| Timeouts | How long should the client wait before treating an operation as too slow? |
| Retries | Should the client try again after a temporary failure, how many times, and with what wait? |
| Reliability | How do we make the whole workflow predictable when individual parts fail? |

They are related but not interchangeable. A timeout tells you an operation did **not finish in time**; it does not tell you whether the server completed it in the background. That matters enormously for writes (issuing a payment, updating a claim).

## 10.2 The failure flow

```
Without vs with error handling
Claims AI Agent -> MCP Client -- call_tool("get_claim") --> Claims MCP Server
                                                              -> Claims API  X  unavailable

With handling:
Call get_claim -> success? --Yes--> return claim
                     |
                     No -> transient? --Yes--> retry with bounded backoff
                                                -> success, or retries exhausted
                                       --No---> return a meaningful error (no retry)
```

The key decision is **transient or permanent**.

| Failure | Typical response |
|---|---|
| Temporary network failure | Retry within limits |
| Service temporarily unavailable | Retry with backoff |
| Invalid claim id | Return an error; do not retry unchanged input |
| Missing authorisation | Deny; do not retry with the same credentials |
| Tool exceeds its deadline | Stop waiting; decide whether a retry is safe |
| Unknown server error | Record diagnostics; retry only if classified as safe |

> **TIP** Classify errors by real exception types, protocol responses and downstream HTTP status codes (such as 429, 502, 503, 504), not by the wording of an error message.

## 10.3 Retries with exponential backoff and jitter (tested)

```python
109_retry_demo.py
import asyncio
import random


class TransientServiceError(Exception):
    """A temporary failure that may succeed on retry."""


class PermanentServiceError(Exception):
    """A failure that retrying will not fix."""


_attempts = {"count": 0}


async def get_claim_from_service(claim_id: str) -> dict:
    """Simulate a claims API that fails twice before succeeding."""
    _attempts["count"] += 1
    attempt = _attempts["count"]
    print(f"[API] Attempt {attempt} for {claim_id!r}")

    if claim_id == "":
        raise PermanentServiceError("claim_id is required")
    if attempt <= 2:
        raise TransientServiceError("Claims API temporarily unavailable")
    return {"claim_id": claim_id, "status": "OPEN", "estimated_loss": 25000}


async def call_with_retry(func, *args, max_attempts=3, base_delay=1.0):
    """Retry only TRANSIENT errors, with exponential backoff + jitter."""
```

```
    for attempt in range(1, max_attempts + 1):
        try:
            return await func(*args)
        except PermanentServiceError as exc:
            print(f"[RETRY] Permanent error; stopping ({exc})")
            raise
        except TransientServiceError as exc:
            if attempt == max_attempts:
                print("[RETRY] Retries exhausted")
                raise
            delay = base_delay * (2 ** (attempt - 1))      # 1s, 2s, 4s ...
            delay += random.uniform(0, 0.1 * delay)         # jitter
            print(f"[RETRY] Temporary failure ({exc}). Waiting {delay:.2f}s")
            await asyncio.sleep(delay)


async def main():
    try:
        claim = await call_with_retry(get_claim_from_service, "CLM001", max_attempts=3)
        print("[SUCCESS]", claim)
    except Exception as exc:
        print(f"[FAILED] {type(exc).__name__}: {exc}")

    # Permanent error: no retry happens
    _attempts["count"] = 0
    try:
        await call_with_retry(get_claim_from_service, "", max_attempts=3)
    except Exception as exc:
        print(f"[FAILED] {type(exc).__name__}: {exc}")


if __name__ == "__main__":
    asyncio.run(main())
```

```
Run
python l09_retry_demo.py
```

```
OUTPUT  retry demo
[API] Attempt 1 for 'CLM001'
[RETRY] Temporary failure (Claims API temporarily unavailable). Waiting 1.05s
[API] Attempt 2 for 'CLM001'
[RETRY] Temporary failure (Claims API temporarily unavailable). Waiting 2.19s
[API] Attempt 3 for 'CLM001'
[SUCCESS] {'claim_id': 'CLM001', 'status': 'OPEN', 'estimated_loss': 25000}
[API] Attempt 1 for ''
[RETRY] Permanent error; stopping (claim_id is required)
[FAILED] PermanentServiceError: claim_id is required
```

The delay roughly doubles each time: d(n) = d(0) x 2^(n-1), where d(0) is the initial delay and n is the attempt number. **Jitter** adds a small random variation so many clients do not all retry at the same instant after a shared outage.

## Trace

1. Attempt 1: the simulated API raises `TransientServiceError`.
2. Retry decision: classified as transient, so another attempt is scheduled.
3. Backoff: about 1 second, then about 2 seconds.
4. Attempt 2 also fails; the client waits longer.
5. Attempt 3 succeeds and the claim is returned.

### Modify it

Calling it with an empty claim id raises `PermanentServiceError`. There is **no retry**: the second block of output shows `Permanent error; stopping`. A missing claim id is an input problem that waiting cannot fix.

> **NOTE** The demo keeps an attempt counter only to make the failure reproducible. Production code reacts to real service responses and must not keep mutable state like this on a shared function.

## 10.4 Timeouts (tested)

Retries alone are not enough: if the claims API accepts a request but never answers, the client could wait forever. `asyncio.wait_for()` limits how long the caller waits.

```
l09_timeout_demo.py
import asyncio


async def slow_claim_service():
    print("[API] Request started")
    await asyncio.sleep(10)                  # simulate a very slow service
    return {"claim_id": "CLM001", "status": "OPEN"}


async def main():
    try:
        result = await asyncio.wait_for(slow_claim_service(), timeout=2)
        print("[SUCCESS]", result)
    except TimeoutError:
        print("[TIMEOUT] Claims API exceeded 2 seconds")


if __name__ == "__main__":
    asyncio.run(main())
```

```
OUTPUT  timeout demo
[API] Request started
[TIMEOUT] Claims API exceeded 2 seconds
```

Internally: the coroutine starts; `wait_for` waits; the deadline expires; `TimeoutError` is raised; control returns to the caller instead of blocking for the full 10 seconds.

> **WATCH OUT** Cancelling a local coroutine does **not** guarantee the remote server or database operation stopped. For reads, retrying is usually fine. For **writes**, a timeout leaves the outcome unknown; before retrying use an **idempotency key** or check the operation's status, otherwise you may apply it twice (for example two payments).

## 10.5 Applying both to a real MCP client call (tested)

The wrapper below puts a timeout and a bounded retry around `session.call_tool()`. Note two rules: `isError=True` is a tool-level failure and is **not retried blindly** (an invalid claim id will not become valid on the second try), and only transient exception types are retried.

```
l09_mcp_reliable_call.py
import asyncio
import random
import sys
from pathlib import Path

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
```

```
class PermanentToolError(Exception):
    """The tool reported an error that retrying will not fix."""


async def call_mcp_tool_reliably(session: ClientSession, tool_name: str,
                                 arguments: dict, *, timeout_seconds: float = 5,
                                 max_attempts: int = 3, base_delay: float = 0.5):
    """Timeout + bounded retry (backoff + jitter) around session.call_tool()."""
    for attempt in range(1, max_attempts + 1):
        try:
            result = await asyncio.wait_for(
                session.call_tool(tool_name, arguments), timeout=timeout_seconds)

            # isError=True is a tool-level failure. Do NOT blindly retry it:
            # an invalid claim id will not become valid on the second try.
            if result.isError:
                raise PermanentToolError(result.content[0].text)
            return result

        except PermanentToolError:
            raise
        except (TimeoutError, ConnectionError, OSError) as exc:   # transient
            if attempt == max_attempts:
                raise
            delay = base_delay * (2 ** (attempt - 1))
            delay += random.uniform(0, 0.1 * delay)
            print(f"[RETRY] attempt {attempt} failed ({type(exc).__name__}); "
                  f"waiting {delay:.2f}s")
            await asyncio.sleep(delay)


async def main():
    server = Path(__file__).with_name("l05_server.py")
    params = StdioServerParameters(command=sys.executable, args=[str(server)])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            result = await call_mcp_tool_reliably(
                session, "get_claim", {"claim_id": "CLM001"})
            print("[OK]", result.content[0].text.replace("\n", " "))

            try:      # schema violation -> isError=True -> permanent, no retry
                await call_mcp_tool_reliably(
                    session, "calculate_reserve",
                    {"claim_id": "CLM001", "severity_buffer": "lots"})
            except PermanentToolError as exc:
                print("[PERMANENT]", str(exc).splitlines()[0])


if __name__ == "__main__":
    asyncio.run(main())
```

```
OUTPUT  wrapper against the real claims server
[OK] {   "claim_id": "CLM001",   "policy_id": "POL1001",   "status": "OPEN",   "loss_type": "Windshield Dama
    ge",   "estimated_amount": 18500.0 }
[PERMANENT] Error executing tool calculate_reserve: 1 validation error for calculate_reserveArguments
```

**TESTED** The first call succeeds. The second sends a string where a number is required; the server's schema validation returns `isError=True`, the wrapper treats it as permanent and stops without retrying.

Why this matters for an agent: a LangGraph workflow that calls an MCP tool can have the retry wrapper absorb a transient failure before control returns to the graph. But **retries at several layers multiply**: if the

MCP client and the underlying HTTP client each retry 3 times, one agent action can cause up to 9 downstream attempts. Define a retry budget and decide which layer owns retries.

A fuller production version distinguishes MCP protocol errors from tool execution errors, transport disconnections from transient network failures, timeouts from unknown outcomes, validation and authorisation failures, and downstream HTTP status codes.

## 10.6 Circuit breaker (tested)

If the claims service is down for 15 minutes and every request keeps retrying, the MCP server wastes resources calling something that cannot answer. A **circuit breaker** temporarily stops calls to a failing dependency. It has three states:

| State | Meaning | Behaviour |
|---|---|---|
| CLOSED | Normal operation | Requests pass through; failures are counted |
| OPEN | Dependency looks unhealthy | Calls are rejected immediately instead of hitting the failing service |
| HALF-OPEN | Recovery test | After a cooldown a limited test request is allowed; success closes the circuit, failure re-opens it |

Do not confuse the four tools:

| Mechanism | What it does |
|---|---|
| Retry | Tries the same operation again after a failure |
| Circuit breaker | Stops calling an unhealthy dependency |
| Timeout | Stops waiting beyond a deadline |
| Rate limit | Restricts how many requests a caller may make in a time window |

```
109_circuit_breaker.py
import time


class CircuitOpenError(Exception):
    pass


class SimpleCircuitBreaker:
    """CLOSED -> (failures >= threshold) -> OPEN -> (cooldown) -> HALF-OPEN."""

    def __init__(self, failure_threshold=3, reset_after=10):
        self.failure_threshold = failure_threshold
        self.reset_after = reset_after
        self.failures = 0
        self.opened_at = None

    def before_call(self):
        if self.opened_at is None:
            return                                  # CLOSED: allow
        elapsed = time.monotonic() - self.opened_at
        if elapsed < self.reset_after:
            raise CircuitOpenError("Circuit is open")   # OPEN: fail fast
        print("[CIRCUIT] Cooldown ended; testing recovery (HALF-OPEN)")
        self.opened_at = None
        self.failures = self.failure_threshold - 1   # one more failure re-opens

    def record_success(self):
        self.failures = 0
        self.opened_at = None
```

```
    def record_failure(self):
        self.failures += 1
        print(f"[CIRCUIT] Failure count: {self.failures}")
        if self.failures >= self.failure_threshold:
            self.opened_at = time.monotonic()
            print("[CIRCUIT] OPENED")


def flaky_service():
    raise ConnectionError("Claims API down")


if __name__ == "__main__":
    breaker = SimpleCircuitBreaker(failure_threshold=3, reset_after=2)

    for _ in range(4):
        try:
            breaker.before_call()
            flaky_service()
        except CircuitOpenError as exc:
            print("[REJECTED]", exc)
        except ConnectionError:
            breaker.record_failure()

    time.sleep(2.1)                                    # wait for the cooldown
    try:
        breaker.before_call()
        print("[SERVICE] recovered")
        breaker.record_success()
    except CircuitOpenError as exc:
        print("[REJECTED]", exc)
```

```
OUTPUT  circuit breaker demo
[CIRCUIT] Failure count: 1
[CIRCUIT] Failure count: 2
[CIRCUIT] Failure count: 3
[CIRCUIT] OPENED
[REJECTED] Circuit is open
[CIRCUIT] Cooldown ended; testing recovery (HALF-OPEN)
[SERVICE] recovered
```

**NOTE** This is a simplified single-threaded demonstration of the state transitions. A production breaker also needs concurrency control, a rolling failure window, a proper cooldown and careful handling of simultaneous requests in HALF-OPEN.

## 10.7 Making the whole workflow reliable

```
A three-stage claims workflow
Claim intake -> Get Claim (MCP) -> Get Policy (MCP) -> Calculate Reserve (MCP) -> Save recommendation
```

| Stage | Failure strategy |
|---|---|
| Get claim | Bounded retry for transient read failures |
| Get policy | Retry safely, within the overall deadline |
| Calculate reserve | Validate inputs; return a meaningful failure |
| Save recommendation | Transaction and idempotency safeguards |
| Entire workflow | Track an overall deadline; record which stages succeeded |

Four production rules:

1. **Use an overall deadline.** Three 5-second timeouts do not guarantee the workflow ends within 5 seconds; give the whole workflow its own deadline.
2. **Make retries safe.** Reads are easy to repeat; for writes use an idempotency key or query the operation's status first.
3. **Log useful context.** Tool name, request/correlation id, attempt number, duration, error category, outcome. Never log access tokens or unnecessary claim PII.
4. **Return meaningful failures.** If claim retrieval fails, do not fabricate details or let the LLM invent a replacement. Say the source is unavailable and whether retry or human review is appropriate.

Track: latency per tool, error rate, retry rate, timeout rate, circuit-breaker state. These show whether the problem is in the MCP client, the MCP server or the underlying claims system.

## 10.8 Where does each failure occur? (layers)

| Layer | Responsibilities |
|---|---|
| 1. MCP client | Request deadlines, transport failures, bounded retries, connection management |
| 2. MCP server | Validate tool arguments, enforce authorisation, execute the tool, return appropriate errors |
| 3. Downstream service | Its own timeouts, transactions, rate limits and business errors |
| 4. Agent orchestration | LangGraph (or another orchestrator) decides whether to continue, retry a node, use an allowed fallback or stop |

MCP does not automatically implement every reliability mechanism; you design them at the right layer. A LangGraph node retry and an MCP client retry are different things; if both retry independently they multiply requests and add load.

## 10.9 Interview questions on reliability

### Q. What is exponential backoff?

A retry strategy where the delay between attempts grows exponentially, commonly doubling after each failure. Jitter adds randomness so clients do not retry in lockstep.

### Q. Should we retry every MCP tool error?

No. Retry only failures classified as transient and only when the operation is safe to repeat. Validation errors, authorisation failures and most permanent business errors must not be retried unchanged.

### Q. What happens if a tool times out after successfully updating a claim?

The client cannot know whether the update succeeded. Blindly repeating it could duplicate the effect. Use idempotency keys, transactional safeguards or a status-check operation.

### Q. What is a circuit breaker and how does it differ from retry?

Retries attempt recovery by repeating an operation. A circuit breaker temporarily blocks calls to an unhealthy dependency so it can recover and so you stop wasting resources.

### Q. Where should retries be implemented in an MCP architecture?

At the layer that understands the failure and can safely repeat the operation. Avoid overlapping retry policies without an explicit retry budget.

### Q. How would you make your claims MCP server production-ready?

Bounded retries with exponential backoff and jitter, request and workflow deadlines, error classification, circuit breakers for unstable dependencies, idempotency and transactional safeguards for writes, structured logging, tracing and metrics, and clear failure responses so the agent never invents missing data.

## 10.10 Lesson 9 checklist

- [ ] I can distinguish transient failures from permanent failures.
- [ ] I understand exponential backoff and jitter.
- [ ] I can explain timeouts and why a timeout does not prove a write failed.
- [ ] I understand retries, circuit breakers and rate limits.
- [ ] I can identify which layer should handle a failure.
- [ ] I understand idempotency, observability and workflow deadlines.

# 11. MCP with LangChain and LangGraph

The question: **how does a LangGraph agent use tools exposed by an MCP server, and why do we need both?**

## 11.1 MCP vs LangChain vs LangGraph

| Technology | Role | What it does |
|---|---|---|
| MCP | Integration layer | Standardises how an AI application discovers and communicates with external capabilities |
| LangChain | Model and tool building blocks | Model interfaces, prompts, parsers, tool abstractions, integrations |
| LangGraph | Workflow orchestration | Execution, state, conditional routing, retries, multi-step decisions |

Imagine three AI applications (Claims Triage, Fraud Investigation, Underwriting) that all need the policy system. With MCP they connect to the same compatible server while each keeps its own reasoning and workflow. **MCP does not replace LangGraph, and LangGraph does not replace MCP.**

## 11.2 Architecture: follow a request

User: "Retrieve claim CLM001 and calculate its recommended reserve."

```
One request, five layers
User request
  -> LangGraph agent                        maintains state, controls the workflow
  -> LangChain chat model + MCP adapter     model picks a discovered tool; adapter executes it
  -> Claims MCP server                      get_claim -> calculate_reserve
  -> Claims API / database                  source of claim data
```

Two different decisions occur: the **model decision** (which tool, with what arguments) and the **workflow decision** (what to do with the result and what happens next). The model may call `get_claim`; LangGraph controls the tool-execution cycle.

## 11.3 Build it

```
Project
mcp_langgraph_demo/
  110_claims_server.py       exposes capabilities through MCP
  110_claims_agent.py        discovers them and runs a LangGraph agent
  .env                       Azure OpenAI settings
```

```
Install
pip install mcp langchain langgraph langchain-openai langchain-mcp-adapters python-dotenv
```

| Package | Purpose |
|---|---|
| `mcp` | Build the MCP server |
| `langchain-mcp-adapters` | Convert MCP capabilities into LangChain-compatible tools |
| `langgraph` / `langchain` | Run the agent workflow |
| `langchain-openai` | Connect to an OpenAI-compatible chat model (`AzureChatOpenAI` for Azure) |
| `python-dotenv` | Load environment variables |

## Step 1: the MCP server

The server has no LangGraph and no LLM; it only exposes capabilities. Reserve example: 25,000 x (1 + 0.15) = 28,750 (educational formula).

```
l10_claims_server.py
import sys
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Claims MCP Server")

CLAIMS = {
    "CLM001": {"claim_id": "CLM001", "status": "OPEN",
               "estimated_loss": 25000, "severity": "High"},
    "CLM002": {"claim_id": "CLM002", "status": "UNDER_REVIEW",
               "estimated_loss": 12000, "severity": "Medium"},
}

SEVERITY_BUFFER = {"Critical": 0.20, "High": 0.15, "Medium": 0.10, "Low": 0.05}


@mcp.tool()
def get_claim(claim_id: str) -> dict:
    """Retrieve claim details using a claim ID."""
    print(f"[MCP SERVER] get_claim called: {claim_id}", file=sys.stderr)
    claim = CLAIMS.get(claim_id)
    if claim is None:
        return {"error": "CLAIM_NOT_FOUND", "claim_id": claim_id}
    return claim


@mcp.tool()
def calculate_reserve(estimated_loss: float, severity: str) -> dict:
    """Calculate a preliminary reserve using a severity buffer."""
    print(f"[MCP SERVER] calculate_reserve: {estimated_loss}, {severity}",
          file=sys.stderr)
    buffer = SEVERITY_BUFFER.get(severity)
    if buffer is None:
        return {"error": "INVALID_SEVERITY", "severity": severity,
                "allowed": list(SEVERITY_BUFFER)}
    return {"estimated_loss": estimated_loss, "severity": severity,
            "buffer": buffer,
            "recommended_reserve": round(estimated_loss * (1 + buffer), 2)}


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

**WATCH OUT** On stdio, diagnostic output must go to **stderr**; a `print()` to stdout would corrupt the protocol stream. The code above already does this.

## Step 2: Azure OpenAI settings

```
.env (never commit this file)
AZURE_OPENAI_API_KEY=your_api_key
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_API_VERSION=your_supported_api_version
AZURE_OPENAI_CHAT_DEPLOYMENT=your_deployment_name
```

## Step 3: discover MCP tools as LangChain tools (tested)

l10_discover_tools.py

```
import asyncio
import sys
from pathlib import Path

from langchain_mcp_adapters.client import MultiServerMCPClient

SERVER = str(Path(__file__).with_name("l10_claims_server.py"))


async def main():
    client = MultiServerMCPClient({
        "claims": {
            "transport": "stdio",
            "command": sys.executable,       # same interpreter as the agent
            "args": [SERVER],
        }
    })

    # tools/list -> LangChain-compatible tool objects
    tools = await client.get_tools()
    print("[AGENT] Discovered tools:")
    for tool in tools:
        print(f"- {tool.name}: {tool.description}")

    # Use a discovered tool directly (no LLM involved)
    get_claim = next(t for t in tools if t.name == "get_claim")
    print("\n[DIRECT CALL]", await get_claim.ainvoke({"claim_id": "CLM001"}))

    calc = next(t for t in tools if t.name == "calculate_reserve")
    print("[DIRECT CALL]", await calc.ainvoke(
        {"estimated_loss": 25000, "severity": "High"}))


if __name__ == "__main__":
    asyncio.run(main())
```

OUTPUT python l10_discover_tools.py

```
[AGENT] Discovered tools:
- get_claim: Retrieve claim details using a claim ID.
- calculate_reserve: Calculate a preliminary reserve using a severity buffer.

[DIRECT CALL] [{'type': 'text', 'text': '{  "claim_id": "CLM001",  "status": "OPEN",  "estimated_loss": 2500
    0,  "severity": "High"}'}]
[DIRECT CALL] [{'type': 'text', 'text': '{  "estimated_loss": 25000.0,  "severity": "High",  "buffer": 0.15,
      "recommended_reserve": 28750.0}'}]
```

**TESTED** `client.get_tools()` returned LangChain tool objects built from the MCP schemas, and invoking one called the MCP server over stdio. No manual wrapper per tool was written.

## Step 4: the LangGraph agent with Azure OpenAI

l10_claims_agent.py

```
import asyncio
import os
import sys
from pathlib import Path

from dotenv import load_dotenv
from langchain_openai import AzureChatOpenAI
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent
```

```
load_dotenv()
SERVER = str(Path(__file__).with_name("l10_claims_server.py"))


async def main():
    # 1. MCP client configuration (the adapter launches the stdio server)
    client = MultiServerMCPClient({
        "claims": {"transport": "stdio", "command": sys.executable,
                   "args": [SERVER]}
    })

    # 2. Discover MCP tools as LangChain-compatible tools
    tools = await client.get_tools()
    print("[AGENT] Discovered tools:", [t.name for t in tools])

    # 3. Azure OpenAI chat model
    model = AzureChatOpenAI(
        azure_deployment=os.environ["AZURE_OPENAI_CHAT_DEPLOYMENT"],
        api_version=os.environ["AZURE_OPENAI_API_VERSION"],
        temperature=0,
    )

    # 4. Agent = LangGraph loop: model node <-> tools node
    agent = create_agent(model, tools)

    # 5. Run
    response = await agent.ainvoke({"messages": [(
        "user",
        "Retrieve claim CLM001 and calculate its recommended reserve. "
        "Use the tools; do not do the arithmetic yourself.")]})

    for message in response["messages"]:
        message.pretty_print()


if __name__ == "__main__":
    asyncio.run(main())
```

**VERSION NOTE** The study chat used `langgraph.prebuilt.create_react_agent`. In the tested versions (`langgraph` 1.2.17, `langchain` installed alongside) that function is **deprecated** and the replacement is `from langchain.agents import create_agent`; the code above uses the new one. Both work today; the old one prints a deprecation warning.

**NOTE** This file needs real Azure OpenAI credentials, so the LLM call itself was **not run** here. Everything else was, as the next step shows.

## Step 5: run the whole loop without an LLM key (tested)

A scripted fake model plays the role of the LLM: it "decides" to call `get_claim`, then `calculate_reserve`, then answers. Tool discovery, the agent loop and the MCP stdio calls are all real. This is a handy technique to test agent plumbing without spending tokens.

**l10_agent_scripted_test.py**

```
"""Run the LangGraph + MCP loop WITHOUT an LLM key.

A scripted fake model plays the part of the LLM: it "decides" to call
get_claim, then calculate_reserve, then answers. Everything else (tool
discovery, the LangGraph loop, MCP stdio calls) is the real thing.
"""
import asyncio
import sys
```

```
from pathlib import Path

from langchain_core.language_models.fake_chat_models import GenericFakeChatModel
from langchain_core.messages import AIMessage
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

SERVER = str(Path(__file__).with_name("l10_claims_server.py"))


class ScriptedModel(GenericFakeChatModel):
    def bind_tools(self, tools, **kwargs):      # the agent calls this
        return self


script = iter([
    AIMessage(content="", tool_calls=[
        {"name": "get_claim", "args": {"claim_id": "CLM001"}, "id": "call_1"}]),
    AIMessage(content="", tool_calls=[
        {"name": "calculate_reserve",
         "args": {"estimated_loss": 25000, "severity": "High"}, "id": "call_2"}]),
    AIMessage(content="Claim CLM001 is OPEN (High severity). "
                      "Recommended reserve: 28750.0."),
])


async def main():
    client = MultiServerMCPClient({"claims": {
        "transport": "stdio", "command": sys.executable, "args": [SERVER]}})
    tools = await client.get_tools()
    agent = create_agent(ScriptedModel(messages=script), tools)

    result = await agent.ainvoke({"messages": [("user", "Reserve for CLM001?")]})
    for message in result["messages"]:
        message.pretty_print()


if __name__ == "__main__":
    asyncio.run(main())
```

```
OUTPUT  python l10_agent_scripted_test.py
================================ Human Message =================================

Reserve for CLM001?
================================== Ai Message ==================================
Tool Calls:
  get_claim (call_1)
 Call ID: call_1
  Args:
    claim_id: CLM001
================================= Tool Message =================================
Name: get_claim

[{'type': 'text', 'text': '{  "claim_id": "CLM001",  "status": "OPEN",  "estimated_loss": 25000,  "severity"
    : "High"}'}]
================================== Ai Message ==================================
Tool Calls:
  calculate_reserve (call_2)
 Call ID: call_2
  Args:
    estimated_loss: 25000
    severity: High
================================= Tool Message =================================
Name: calculate_reserve

[{'type': 'text', 'text': '{  "estimated_loss": 25000.0,  "severity": "High",  "buffer": 0.15,  "recommended
    _reserve": 28750.0}'}]
```

```
================================== Ai Message ==================================

Claim CLM001 is OPEN (High severity). Recommended reserve: 28750.0.
```

### Expected flow with a real model

1. **Tool discovery**: the adapter connects to the claims MCP server and retrieves tool definitions.
2. **Model invocation**: LangGraph sends the request and the tool descriptions to the model.
3. **First tool call**: the model requests `get_claim` with `claim_id="CLM001"`.
4. **MCP execution**: the adapter sends the call to the server and receives the claim data.
5. **Second tool call**: the model requests `calculate_reserve` with the returned loss and severity.
6. **Final response**: the model receives the result and explains the calculation (CLM001, loss 25,000, High severity, 15% buffer, reserve 28,750).

The exact wording of the model's answer varies.

## 11.4 Internals: how an MCP tool becomes a LangGraph tool

```
The adapter in the middle
MCP server -- tools/list --> tool name + description + input schema
   -> langchain-mcp-adapters -> LangChain-compatible tool object
   -> LangGraph agent -- model requests a tool call --> adapter sends tools/call -> server
   -> tool result returns to the agent
```

The adapter maps MCP tool definitions to LangChain tools, translates tool calls and arguments into MCP requests, returns results to the agent and manages the connection. It does **not** make the model infallible: the model can still pick an unsuitable tool or produce invalid arguments, and the server must still validate.

LangGraph manages the tool-calling loop:

```
The loop
START -> Model node -> tool requested? --No--> END
                           |
                           Yes -> Tools node -> back to Model node
```

LangGraph coordinates the sequence; MCP standardises access to the capabilities.

## 11.5 A second AI application reusing the same server

The fraud application can call the same server (here by selecting a discovered tool and invoking it directly):

```
Reuse without re-implementing
client = MultiServerMCPClient({"claims": {"transport": "stdio",
                                          "command": sys.executable,
                                          "args": ["l10_claims_server.py"]}})
tools = await client.get_tools()
get_claim_tool = next(t for t in tools if t.name == "get_claim")
result = await get_claim_tool.ainvoke({"claim_id": "CLM001"})
```

> **NOTE** With **stdio**, each application launches its **own** server process; they share the code and capability definitions, not one running instance. With a remote **HTTP** server, many compatible clients connect to the same deployed service, subject to authentication, authorisation and capacity limits. The snippet above is a direct tool call, not yet a full Fraud Investigation Agent; add a model and workflow if it needs to reason.

### What if the claim does not exist?

Calling with `CLM999` returns `{"error": "CLAIM_NOT_FOUND", "claim_id": "CLM999"}`. That is a structured result, **not** a Python exception, so an agent might continue and pass missing data to the reserve tool. For a robust workflow, instruct the agent to stop on tool errors and, for financial calculations, **validate returned fields in application code** instead of relying on the model.

## 11.6 Production considerations

| Concern | Where to handle it |
|---|---|
| Tool discovery and MCP communication | MCP client / adapter |
| Reasoning and workflow order | LangGraph |
| Prompts and model configuration | LangChain / model integration |
| Input validation and business rules | MCP server and underlying service |
| Identity and authorisation | MCP server, identity provider and backend |
| Retry budgets and workflow deadlines | Client, server and orchestration layers as appropriate |
| Financial calculations | Deterministic, validated business logic |

1. **Do not trust model-generated arguments.** Validate claim ids, numeric ranges, severities and so on at the server boundary.
2. **Keep business logic out of the prompt.** A reserve calculation must be deterministic code, not LLM arithmetic.
3. **Handle failures explicitly.** A missing claim stops dependent calculations unless an approved fallback exists.
4. **Least privilege.** A fraud agent that only reads claims must not receive write or payment tools.
5. **Avoid duplicate retries** between adapter, MCP server, downstream API and LangGraph.

If your application only needs a local Python function, MCP may be unnecessary. It pays off when you need a standardised interface that compatible AI applications can discover and consume.

## 11.7 Interview questions

### Q. Why integrate MCP with LangGraph instead of using LangGraph tools directly?

LangGraph tools are enough when the capabilities are local to one application. MCP is useful when capabilities must be exposed through a standard interface and reused by several compatible AI applications.

### Q. How does langchain-mcp-adapters help?

It converts MCP-exposed capabilities into LangChain-compatible tools so a LangGraph agent can use them without hand-implementing the protocol calls per tool.

### Q. Does MCP decide which tool the LLM calls?

No. MCP exposes and communicates with capabilities. The model chooses a tool from its instructions and the tool descriptions, and the orchestrator manages the execution loop.

### Q. Can two agents use the same MCP server?

Yes. With stdio each may launch its own process; with a remote transport they connect to the same deployed service.

### Q. What happens when an MCP tool returns an error?

The adapter returns it according to its SDK mapping and the orchestration layer decides what happens next. The application should classify failures, block unsafe follow-up actions and never invent missing data.

### Q. Is MCP required to use LangGraph?

No. LangGraph can use plain Python functions, LangChain tools and APIs. MCP is an optional standardised integration mechanism.

## 11.8 Lesson 10 checklist

- [ ] I can distinguish MCP, LangChain and LangGraph responsibilities.
- [ ] I understand how MCP tools are discovered and adapted.
- [ ] I can trace a model tool call through LangGraph to an MCP server.
- [ ] I understand how multiple applications can reuse MCP capabilities.
- [ ] I know why server-side validation and explicit error handling are necessary.

# 12. Tools, Resources and Prompts in One AI Workflow

The question: **when should an agent use a Tool, read a Resource or retrieve a Prompt, and how do they work together?**

Scenario: an adjuster asks "Analyze claim CLM001, check the relevant policy information, and identify any issues requiring human review." The workflow needs three different capabilities.

## 12.1 The three primitives, again

| Primitive | Example | Meaning |
|---|---|---|
| Tool | `get_claim("CLM001")`, `calculate_reserve(25000, "High")` | The client invokes a capability to retrieve data or execute an operation |
| Resource | `policies://POL001` | The client reads content identified by a URI; reading it does not put it in the model's context |
| Prompt | `claim_analysis(claim_details, policy_details, reserve_details)` | The client requests a parameterised prompt the host can use when invoking the model |

A useful memory aid: Tool answers "what can the application **do**?", Resource "what can it **read**?", Prompt "what reusable **instructions** can it use?" The distinction is about the **interface**, not the Python implementation.

## 12.2 Architecture

```
A controlled workflow (the application orchestrates)
User asks for claim analysis
  1. MCP client calls a Tool         get_claim("CLM001")
  2. MCP client reads a Resource     policies://POL001
  3. MCP client calls a Tool         calculate_reserve(25000, "High")
  4. MCP client gets a Prompt        claim_analysis(claim + policy + reserve)
  5. HOST sends the prompt to the LLM
  6. Claim analysis for the adjuster
```

The application explicitly orchestrates the calls here; the LLM does not choose which primitive to use. That is deliberate: in insurance workflows certain steps are mandatory and must not be skipped because a model decided they were unnecessary. In a more autonomous LangGraph agent the model may choose among tools, while graph logic enforces required steps and handles results.

## 12.3 The server: all three primitives (tested)

```
l11_server.py
import json
import sys
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Claims Analysis Server")

CLAIMS = {
    "CLM001": {"claim_id": "CLM001", "policy_id": "POL001",
               "loss_type": "Vehicle damage", "estimated_loss": 25000,
               "severity": "High", "status": "OPEN"},
}
POLICIES = {
```

```
    "POL001": {"policy_id": "POL001", "coverage_type": "Comprehensive motor",
               "coverage_limit": 500000, "deductible": 5000, "status": "ACTIVE"},
}
SEVERITY_BUFFER = {"Critical": 0.20, "High": 0.15, "Medium": 0.10, "Low": 0.05}


# --- TOOLS: things the application can DO ------------------------------
@mcp.tool()
def get_claim(claim_id: str) -> dict:
    """Retrieve claim details using a claim ID."""
    print(f"[MCP SERVER] get_claim({claim_id})", file=sys.stderr)
    return CLAIMS.get(claim_id) or {"error": "CLAIM_NOT_FOUND", "claim_id": claim_id}


@mcp.tool()
def calculate_reserve(estimated_loss: float, severity: str) -> dict:
    """Calculate a preliminary reserve using a severity buffer."""
    buffer = SEVERITY_BUFFER.get(severity)
    if buffer is None:
        return {"error": "INVALID_SEVERITY", "severity": severity}
    return {"estimated_loss": estimated_loss, "severity": severity,
            "buffer": buffer,
            "recommended_reserve": round(estimated_loss * (1 + buffer), 2)}


# --- RESOURCE: information the application can READ ---------------------
@mcp.resource("policies://{policy_id}")
def read_policy(policy_id: str) -> str:
    """Read policy information using a policies:// URI."""
    policy = POLICIES.get(policy_id)
    if policy is None:
        return json.dumps({"error": "POLICY_NOT_FOUND", "policy_id": policy_id})
    return json.dumps(policy, indent=2)


# --- PROMPT: reusable INSTRUCTIONS the application can use --------------
@mcp.prompt()
def claim_analysis(claim_details: str, policy_details: str,
                   reserve_details: str) -> str:
    """Reusable prompt for analysing a claim against its policy."""
    return f"""You are an insurance claims analyst assisting a human adjuster.

CLAIM DETAILS:
{claim_details}

POLICY DETAILS:
{policy_details}

RESERVE CALCULATION (computed by deterministic code - do not recompute):
{reserve_details}

Instructions:
1. Summarize the claim, policy and reserve using ONLY the data above.
2. List coverage questions that need human review. An ACTIVE policy does not
   by itself prove that this loss is covered - do not conclude coverage.
3. List missing information.
4. Do not invent facts. Separate facts from assumptions.

Use the headings: Claim Summary, Coverage Questions, Missing Information,
Items Requiring Human Review.
"""


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Notes: `get_claim` and `calculate_reserve` are **Tools**; `policies://{policy_id}` is a **Resource** (a template; in production the handler would query a database and enforce access control); `claim_analysis` is a **Prompt** with three inputs. The prompt does not execute the LLM; it returns reusable instructions for the host.

## 12.4 The host application (tested with --dry-run)

```
l11_workflow.py
"""Host application: Tool + Resource + Prompt -> LLM.

Run:  python l11_workflow.py --dry-run     (no LLM; prints the final prompt)
      python l11_workflow.py               (needs Azure OpenAI values in .env)
"""
import asyncio
import json
import os
import sys
from pathlib import Path

from dotenv import load_dotenv
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

load_dotenv()
SERVER = Path(__file__).with_name("l11_server.py")


def parse_tool_result(result) -> dict:
    """MCP result -> dict. Stops the workflow on any error."""
    if result.isError:
        raise RuntimeError(f"MCP tool execution failed: {result.content[0].text}")
    data = json.loads(result.content[0].text)
    if "error" in data:                       # business error returned as data
        raise RuntimeError(f"Tool reported a business error: {data}")
    return data


async def main(dry_run: bool):
    params = StdioServerParameters(command=sys.executable, args=[str(SERVER)])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # Discovery of all three primitives
            tools = await session.list_tools()
            templates = await session.list_resource_templates()
            prompts = await session.list_prompts()
            print("[CLIENT] Tools:", [t.name for t in tools.tools])
            print("[CLIENT] Resource templates:",
                  [t.uriTemplate for t in templates.resourceTemplates])
            print("[CLIENT] Prompts:", [p.name for p in prompts.prompts])

            # 1. TOOL: claim -> stop if it does not exist
            claim = parse_tool_result(await session.call_tool(
                "get_claim", {"claim_id": "CLM001"}))

            # 2. RESOURCE: policy, found through the claim's policy_id
            policy_res = await session.read_resource(f"policies://{claim['policy_id']}")
            policy = json.loads(policy_res.contents[0].text)
            if "error" in policy:
                raise RuntimeError(f"Policy unavailable: {policy}")

            # 3. TOOL: deterministic reserve calculation (NOT done by the LLM)
            reserve = parse_tool_result(await session.call_tool(
                "calculate_reserve",
                {"estimated_loss": claim["estimated_loss"],
```

```
                "severity": claim["severity"]}))

            # 4. PROMPT: reusable instructions, filled with our data
            prompt = await session.get_prompt("claim_analysis", {
                "claim_details": json.dumps(claim, indent=2),
                "policy_details": json.dumps(policy, indent=2),
                "reserve_details": json.dumps(reserve, indent=2)})
            prompt_text = prompt.messages[0].content.text

    # 5. The HOST (not MCP) assembles context and calls the LLM
    if dry_run:
        print("\n========== FINAL PROMPT THAT WOULD GO TO THE LLM ==========")
        print(prompt_text)
        return

    from langchain_core.messages import HumanMessage
    from langchain_openai import AzureChatOpenAI
    model = AzureChatOpenAI(
        azure_deployment=os.environ["AZURE_OPENAI_CHAT_DEPLOYMENT"],
        api_version=os.environ["AZURE_OPENAI_API_VERSION"], temperature=0)
    answer = await model.ainvoke([HumanMessage(content=prompt_text)])
    print("\n========== LLM ANALYSIS ==========")
    print(answer.content)


if __name__ == "__main__":
    asyncio.run(main(dry_run="--dry-run" in sys.argv))
```

**Run**

```
python l11_workflow.py --dry-run     # no LLM: prints the final prompt that would be sent
python l11_workflow.py               # needs Azure OpenAI values in .env
```

**OUTPUT** `python l11_workflow.py --dry-run`

```
[CLIENT] Tools: ['get_claim', 'calculate_reserve']
[CLIENT] Resource templates: ['policies://{policy_id}']
[CLIENT] Prompts: ['claim_analysis']

========== FINAL PROMPT THAT WOULD GO TO THE LLM ==========
You are an insurance claims analyst assisting a human adjuster.

CLAIM DETAILS:
{
  "claim_id": "CLM001",
  "policy_id": "POL001",
  "loss_type": "Vehicle damage",
  "estimated_loss": 25000,
  "severity": "High",
  "status": "OPEN"
}

POLICY DETAILS:
{
  "policy_id": "POL001",
  "coverage_type": "Comprehensive motor",
  "coverage_limit": 500000,
  "deductible": 5000,
  "status": "ACTIVE"
}

RESERVE CALCULATION (computed by deterministic code - do not recompute):
{
  "estimated_loss": 25000.0,
  "severity": "High",
  "buffer": 0.15,
  "recommended_reserve": 28750.0
}
```

```
Instructions:
1. Summarize the claim, policy and reserve using ONLY the data above.
2. List coverage questions that need human review. An ACTIVE policy does not
   by itself prove that this loss is covered - do not conclude coverage.
3. List missing information.
4. Do not invent facts. Separate facts from assumptions.

Use the headings: Claim Summary, Coverage Questions, Missing Information,
Items Requiring Human Review.
```

**TESTED** Discovery found 2 tools, 1 resource template and 1 prompt; the claim, policy and reserve were fetched through MCP; the prompt came back with all three data blocks filled in. Only the final LLM call was not run (it needs credentials). An ACTIVE policy does not by itself prove the loss is covered; the prompt tells the model not to conclude that.

### Trace each request

| Client method | MCP operation | What comes back |
|---|---|---|
| `session.list_tools()` | `tools/list` | Tool names, descriptions, input schemas |
| `session.call_tool(...)` | `tools/call` | Tool execution result |
| `session.list_resource_templates()` | `resources/templates/list` | URI templates |
| `session.read_resource(...)` | `resources/read` | Resource contents |
| `session.list_prompts()` | `prompts/list` | Prompt names and arguments |
| `session.get_prompt(...)` | `prompts/get` | Prompt messages |

1. **Discovery** of all three primitives.
2. **Claim retrieval** through a tool; the helper parses the JSON and stops on any error.
3. **Policy retrieval** through a resource read, using the claim's `policy_id`.
4. **Reserve calculation** through a tool, using the claim's loss and severity: 25,000 x 1.15 = 28,750.
5. **Prompt retrieval** with the claim, policy and reserve filled in.
6. **LLM invocation**: the host assembles the context and calls Azure OpenAI.

## 12.5 Modify, break and fix

- **Claim does not exist** (`CLM999`): the server returns `CLAIM_NOT_FOUND`; `parse_tool_result` raises, so the reserve step never runs. This is deliberate: dependent steps must not continue when a required source record is missing.
- **Policy resource unavailable** (wrong URI or backend down): stop the workflow and report the failure. Never substitute an arbitrary policy or let the LLM guess the coverage limit. In production log the failing URI, correlation id and error category without exposing sensitive data.
- **The prompt changes** (the business wants JSON output with keys such as `claim_summary`, `coverage_questions`, `recommended_reserve`, `human_review_required`): update the prompt, but instructions alone do not guarantee valid JSON. If downstream systems depend on it, add structured-output validation in the host and reject or repair invalid output under a controlled policy.

**KEY IDEA** Tools provide capabilities, Resources provide data, Prompts provide reusable instructions. The host remains responsible for coordinating them and validating the final result.

## 12.6 Internals: what actually crosses the MCP boundary

MCP does not automatically combine the three results into an LLM prompt; the host does it explicitly:

```
The host assembles the context
Tool result ------+
Resource content -+--> Host assembles context --> LLM request --> Analysis response
Prompt messages --+
```

This is why reading a resource does not mean the LLM has seen it: the host must deliberately include it.

### The same workflow as a LangGraph graph

```
Nodes and state
START -> Retrieve Claim -> Read Policy Resource -> Calculate Reserve
      -> Retrieve Analysis Prompt -> Run LLM Analysis -> Validate Result -> END
```

Each node writes its result into graph state, which makes intermediate values inspectable and allows conditional edges: if claim retrieval returns `CLAIM_NOT_FOUND`, a conditional edge routes to an error-handling node instead of the reserve calculation. The MCP integration layer stays the same; LangGraph adds explicit orchestration around it.

## 12.7 Production considerations

- **Authorisation**: a user must be authorised to read the claim and policy even if the server exposes them.
- **Data minimisation**: pass the model only what the task needs.
- **Untrusted content**: claim descriptions, uploaded documents and retrieved records may contain malicious instructions; treat them as data, not as trusted system instructions.
- **Deterministic calculations**: financial maths in validated code, not LLM arithmetic.
- **Output validation**: validate the analysis before storing it or triggering downstream actions.
- **Version compatibility**: pin compatible MCP, LangChain and model-integration versions, especially when moving between SDK generations.
- **Fixed vs autonomous**: use fixed graph edges for mandatory controls and conditional routing where flexibility is legitimate. An autonomous agent is more flexible but may skip steps or call unnecessary tools.

## 12.8 Interview questions

### Q. Can an MCP Resource execute business logic?

A resource handler can run code to retrieve or prepare content, but its MCP interface represents readable content identified by a URI, whereas a Tool represents a callable capability.

### Q. Does retrieving an MCP Prompt call the LLM?

No. It returns prompt messages; the host decides whether and when to send them to a model.

### Q. Why not put all business logic in MCP Tools?

Tools can expose business operations, but workflow orchestration, model invocation and state management are separate concerns. Separating them makes the system easier to test and maintain.

### Q. How would you handle a missing claim?

Return or raise a structured error, stop dependent steps, record the failure and report that the required data is unavailable. Do not invent claim or policy details.

### Q. When would you use a fixed workflow instead of an autonomous agent?

When steps are mandatory, ordered or compliance-sensitive. Use conditional routing or agent-driven tool selection when the next action genuinely depends on what was discovered.

### Q. What is the role of LangGraph here?

LangGraph manages state and execution flow: nodes, conditional edges and error-handling branches. MCP provides the standard interface to external capabilities.

## 12.9 Lesson 11 checklist

- [ ] I can distinguish Tools, Resources and Prompts.
- [ ] I understand how the host assembles data and instructions for the LLM.
- [ ] I can trace discovery, tool calls, resource reads and prompt retrieval.
- [ ] I understand why mandatory workflow steps should not depend entirely on model decisions.
- [ ] I can explain how LangGraph orchestrates MCP-backed operations.

# 13. Popular MCP Servers

The ecosystem has many community and first-party servers. These are the ones most often asked about:

| Server | Typical launch | What it does |
|---|---|---|
| Filesystem | `npx -y @modelcontextprotocol/server-filesystem <dir>` | Read, write, move and search files in configured directories. The canonical example server. |
| Git | `uvx mcp-server-git` | Read git repositories: diff, log, blame, show a file at a revision. |
| SQLite | `uvx mcp-server-sqlite` | Query and modify a local SQLite database, including schema inspection. |
| PostgreSQL | `npx -y @modelcontextprotocol/server-postgres <connection-url>` | Read-only access to Postgres: tables as resources, SQL as a tool. |
| Brave Search | `npx -y @modelcontextprotocol/server-brave-search` | Web and local search through the Brave Search API (needs `BRAVE_API_KEY`). |
| GitHub | `npx -y @modelcontextprotocol/server-github` | Issues, pull requests, repository search and file management through the GitHub API. |
| Slack | `npx -y @modelcontextprotocol/server-slack` | Post messages, read channel history, manage users. |
| Memory | `npx -y @modelcontextprotocol/server-memory` | A persistent knowledge graph so the AI can remember facts across sessions. |

**VERSION NOTE** This list was not re-checked against the live repositories. Several of the original reference servers (for example GitHub, Postgres, Slack and Brave Search) have since been moved to an archive in favour of vendor-maintained servers, and the launch commands change. Check the current `modelcontextprotocol/servers` repository before relying on a command.

## 13.1 Configuring servers in Claude Desktop

Claude Desktop reads `claude_desktop_config.json` (on Windows under `%APPDATA%\Claude`). Each entry tells the host **how to launch the server over stdio**; the host spawns it as a child process.

```
claude_desktop_config.json
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "C:\\Users\\admin\\docs"]
    },
    "claims": {
      "command": "C:\\path\\to\\venv\\Scripts\\python.exe",
      "args": ["C:\\path\\to\\l05_server.py"]
    },
    "brave-search": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-brave-search"],
      "env": { "BRAVE_API_KEY": "YOUR_KEY_HERE" }
    }
  }
}
```

**TIP** Use the **full path to the Python interpreter** of the virtual environment that has `mcp` installed, and absolute paths to the script. A bare `python` often resolves to a different interpreter, and that is the usual reason a server "does not start" in a desktop host. Restart the host after editing the file.

**WATCH OUT** Keep secrets in the `env` block of a file only you can read, never in a repository. Remember that a stdio server runs with **your** OS privileges.

# 14. Interview Questions and Answers (Consolidated)

Per-topic questions are at the end of Sections 2, 3, 4, 5, 7, 8, 10, 11 and 12. This section collects the highest-value ones for revision, plus questions that cut across topics.

## 14.1 The 30-second answers

**Q. What is MCP and why was it created?**

MCP, the Model Context Protocol, is an open standard from Anthropic for connecting AI applications to external data and tools. It replaces ad-hoc, per-integration connectors with one standard protocol, which reduces duplication, improves security and enables an ecosystem of reusable servers. It is a protocol, not an LLM framework.

**Q. Explain the three roles in MCP.**

The Host is the end-user AI application that owns the LLM. The Client is a protocol layer inside the host that manages one connection to one server. The Server is an independent process that exposes capabilities: tools, resources and prompts.

**Q. What is the difference between a Resource and a Tool?**

A Resource is read-oriented context identified by a URI (analogous to GET); the application reads it to gain knowledge. A Tool is an action that can have side effects; the model decides to call it to do something. Both can return data; the difference is the interface and intended role.

**Q. What transport does Claude Desktop use for local servers?**

stdio. It spawns each configured server as a child process and exchanges newline-delimited JSON-RPC over stdin/stdout.

**Q. How does capability negotiation work?**

During the initialize handshake the client declares what it supports (sampling, roots, elicitation) and the server replies with what it exposes (tools, resources, prompts, logging). Each side then knows what the other can do. (Newer revisions are moving to a stateless discovery model; check the current spec.)

**Q. What underlying protocol do MCP messages use?**

JSON-RPC 2.0: requests (with an id, expecting a reply), responses (matching id, with result or error) and notifications (no id, no reply).

**Q. How does Sampling work and why is it useful?**

A server asks the host to make an LLM call for it. The host shows the request to the user for approval, runs the model and returns the output. It lets a server reason in a loop without its own API keys.

**Q. What is the difference between stdio and HTTP transports?**

stdio is local: simple, no network, secured by the OS process boundary, one client. HTTP (Streamable HTTP) is for remote servers: many clients, needs TLS and authentication, more setup.

**Q. How do you handle errors in MCP tools?**

Tool-level failures are reported in the tool result (`isError`) so the JSON-RPC layer does not crash; protocol problems (unknown method, malformed request) use JSON-RPC error codes. Business errors returned as data need an explicit check by the client. Retry only transient failures.

### Q. What is a URI template in MCP resources?

A pattern such as `db://table/{tableName}/row/{id}` that the client fills in to address a specific resource. Templates are listed with `resources/templates/list`, not `resources/list`.

### Q. How would you secure an HTTP-based MCP server?

HTTPS, OAuth 2.1 with PKCE, validation of the bearer token on every request, narrow scopes, tool-level and record-level authorisation, schema and business validation, rate limiting and audit logging.

### Q. What is Elicitation and when would you use it?

A 2025 feature where a server asks the end user for more input during a tool call, for example to confirm a destructive action or supply a missing parameter. The client must declare support for it.

### Q. What are MCP Roots?

URIs the client exposes to tell the server which parts of the filesystem or workspace it should operate in, so the server avoids touching anything else.

### Q. How does pagination work in MCP?

List methods accept an optional cursor and return an optional `nextCursor`; the client loops, passing the cursor back until none is returned.

### Q. Can an MCP server connect to another MCP server?

Yes. A server can itself be a client to another server, enabling composition and agentic architectures.

## 14.2 Cross-cutting questions

### Q. We already had LangGraph tools. Why do we need MCP?

MCP did not invent tools; it standardises how capabilities are exposed to and consumed by AI applications. LangGraph tools live inside one application. An MCP server owns the capability and any compatible application, in any framework or language, can discover and use it, so the integration is built once.

### Q. Is MCP better than LangGraph tools?

That is the wrong comparison. They solve different layers. LangGraph tools answer "how does my agent run a tool inside my application"; MCP answers "how can an AI application discover and talk to external capabilities through a standard protocol". They work together: LangGraph orchestrates, MCP is the capability layer.

### Q. Why not just call REST APIs?

You can, and MCP does not make REST obsolete. MCP adds a standard, AI-oriented capability model (tools, resources, prompts), discovery and an adapter layer that hides the backend. An MCP server often calls REST internally.

### Q. Is an MCP server a REST API?

No. It is a logical protocol server that speaks MCP over a transport. It may use REST, SQL or Guidewire underneath, and it does not contain an LLM.

### Q. Does the LLM execute the tool?

No. The LLM produces a tool-call decision with arguments. The host's MCP client sends `tools/call`; the server executes the tool.

### Q. Does MCP convert API data into "a format the AI understands"?

Not by itself. The MCP server developer adapts and normalises the backend response, and MCP gives the application a standard structured interface and discovery. LLMs can already read JSON.

### Q. Protocol vs transport vs tool?

The protocol (MCP/JSON-RPC) defines what messages mean (`tools/call`). The transport (stdio, HTTP) defines how they travel. A tool (`get_claim`) is the capability exposed.

### Q. Does reading a resource or fetching a prompt call the LLM?

No. The host decides what to read, how to put it into the context and when to call the model.

### Q. Why check scope and record ownership?

Scope grants permission to a kind of operation; ownership checks stop one user from touching another user's record.

### Q. What should you never print on a stdio server?

Anything to stdout. It carries the protocol; log to stderr.

### Q. How do you test agent plumbing without an LLM key?

Use a scripted fake chat model that emits the tool calls, keep the real MCP server and LangGraph loop (Section 11.3, Step 5).

## 14.3 Rapid-fire definitions

| Term | One line |
|---|---|
| MCP | Open protocol standardising how AI apps connect to external tools, data and prompts |
| Host | The AI application that owns the LLM and the user experience |
| Client | The connection layer in the host for one server |
| Server | Process exposing tools, resources and prompts |
| Tool | Callable action with a JSON-Schema input |
| Resource | Read-oriented content identified by a URI |
| Resource template | A URI with variables such as `claims://{claim_id}` |
| Prompt | Reusable parameterised message template |
| Sampling | Server asks the host for an LLM completion |
| Roots | URIs scoping where the server works |
| Elicitation | Server asks the user for more input |
| stdio | Local transport over stdin/stdout |
| Streamable HTTP | Modern HTTP transport on a single endpoint |
| JSON-RPC 2.0 | The message format: request, response, notification |
| `isError` | Flag in a tool result marking a tool-level failure |
| Idempotency key | Token that makes a repeated write safe |
| Circuit breaker | Stops calling an unhealthy dependency |

## 14.4 Quick reference cheat sheet

| Item | Value / notes |
|---|---|
| Underlying protocol | JSON-RPC 2.0 |
| Spec revisions (selected) | 2024-11-05 (original), 2025-03-26 (Streamable HTTP, OAuth), later revisions exist; check the current spec |
| Local transport | stdio (stdin/stdout, newline-delimited JSON) |
| Remote transport | Streamable HTTP (single endpoint, commonly `/mcp`); HTTP+SSE is legacy |
| Auth standard | OAuth 2.1 with PKCE; bearer token on every request |
| Core primitives | Resources, Tools, Prompts (server); Sampling, Roots, Elicitation (client) |
| Discovery calls | `tools/list`, `resources/list`, `resources/templates/list`, `prompts/list` |
| Invocation calls | `tools/call`, `resources/read`, `prompts/get` |
| Python SDK | `pip install mcp`; `from mcp.server.fastmcp import FastMCP` |
| JS/TS SDK | `npm install @modelcontextprotocol/sdk` |
| Debug tool | `npx @modelcontextprotocol/inspector <server command>` |
| Desktop config | `claude_desktop_config.json`, key `mcpServers` |
| LangChain bridge | `pip install langchain-mcp-adapters`; `MultiServerMCPClient(...).get_tools()` |

# Appendix A: Study Roadmap and Progress

The guided path followed in the study chat, with where each stage lives in this document:

| Stage | Topic | Status | Section |
|---|---|---|---|
| 1 | MCP mental model | Covered | 1 |
| 2 | Host, Client, Server | Covered | 2 |
| 3 | Protocol and JSON-RPC (concept and code) | Covered | 2.9, 7 |
| 4 | Transports: stdio and HTTP (concept and code) | Covered | 3 |
| 5 | Build a proper MCP server and client; tool lifecycle | Covered | 5, 6, 7.6 |
| 6 | Resources | Covered | 4.2 |
| 7 | Prompts | Covered | 4.4 |
| 8 | Security and authorisation | Covered, now with tested code | 8 |
| 9 | Errors, timeouts, retries, reliability | Covered | 10 |
| 10 | MCP with LangChain and LangGraph | Covered | 11 |
| 11 | Tools, Resources and Prompts in one workflow | Covered | 12 |
| 12 | MCP and RAG (announced as next, not part of the transcript) | **Not covered yet** | n/a |

What Lesson 12 was announced to cover: how an MCP server can expose a **search capability backed by a vector database** (for example treaty or policy search), how the agent consumes the retrieved context, and why MCP itself is not the retrieval algorithm or the vector database. Section 4.2.9 has the short version of that idea.

The roadmap the study chat set out at the start also listed a **Claims MCP Server** build and **production architecture** stage, and a final **interview preparation** stage. The Claims MCP Server build is covered by Sections 5, 11 and 12, the production considerations by 5.11, 10, 11.6 and 12.7, and interview preparation by Section 14.

# Appendix B: Findings From Running the Code

The transcript's code had not been run. Running it showed the following. Everything in this table was observed on `mcp` 1.30.0.

| # | What the transcript implied or omitted | What actually happens | Section |
|---|---|---|---|
| 1 | `resources/list` shows `claims://{claim_id}` | It lists only **static** resources. Templates appear under `resources/templates/list` (`list_resource_templates()`) | 4.2.5 |
| 2 | A tool returning `dict` returns structured data | The data arrives as **JSON text** in `result.content`; `structuredContent` is `None` unless the tool has a typed output schema | 3.4, 6.4 |
| 3 | Errors are "errors" | A business error returned as data has `isError=False`. Wrong argument types and unknown tools have `isError=True` | 4.3 |
| 4 | Prompt arguments | Always strings; a returned `str` becomes one `user` message | 4.4.4 |
| 5 | Code examples used a newer SDK generation in places (`MCPServer`, a URL-based `Client`) alongside the older `FastMCP` | Pick one generation. This document uses 1.30.0: `FastMCP`, `ClientSession`, `streamablehttp_client` | 3.5, 5.1 |
| 6 | `create_react_agent` from `langgraph.prebuilt` | Deprecated; use `from langchain.agents import create_agent` | 11.3 |
| 7 | `print()` in a server | Fine on HTTP, **breaks stdio**; log to stderr | 3.4 |
| 8 | Code was cut off mid-file in the pasted transcript | Completed and tested (`l03`, `l04`, `l05`, `l09`, `l10`, `l11`) | all |
| 9 | Lesson 8 (security) coverage was partial | Added a real token-verifying HTTP server and record-level checks, tested | 8.4 |
| 10 | "Read the claim, then compute the reserve in the LLM" is a tempting shortcut | Keep deterministic maths in tools; the LLM only explains | 12 |

Corrections made to the **first version** of these notes:

| Original issue | Fix |
|---|---|
| Arrows, bullets and emoji printed as black boxes (font lacked the glyphs) | Unicode fonts throughout; ASCII diagrams |
| Code lost its indentation (text was set as normal paragraphs) | Proper monospaced blocks that keep spaces and split across pages |
| Low-level resource example used a type that does not exist (`BlobContent`) | Rewritten with `FastMCP` and tested |
| Progress and logging snippets were not real SDK calls | Replaced with `Context.report_progress` and `ctx.info`, tested |
| Some popular-server package names and the Git server launch command were wrong | Corrected, plus a note that the list changes |
| No page numbers in the index | Clickable index with page numbers |

# Appendix C: Code Files and How to Run Them

Every file below is in the `code` folder next to this PDF. Set up once:

```
Setup
python -m venv venv
venv\Scripts\activate
pip install mcp langchain langgraph langchain-openai langchain-mcp-adapters python-dotenv httpx
```

| File | Shows | Run | Verified |
|---|---|---|---|
| `l03_jsonrpc_demo.py` | A hand-written JSON-RPC dispatcher | `python l03_jsonrpc_demo.py` | Yes |
| `l04_stdio_server.py` / `l04_stdio_client.py` | stdio server and the client that launches it | `python l04_stdio_client.py` | Yes |
| `l04_http_server.py` / `l04_http_client.py` / `l01_fraud_ai.py` | Streamable HTTP; two apps, one server | server in one terminal, then each client | Yes |
| `l05_server.py` / `l05_client.py` | Four tools, schemas, three failure kinds | `python l05_client.py` | Yes |
| `l06_server.py` / `l06_client.py` | Resources, templates, static resource, a tool | `python l06_client.py` | Yes |
| `l07_server.py` / `l07_client.py` | Prompts | `python l07_client.py` | Yes |
| `l08_authz_demo.py` | Scopes and record-level checks in plain Python | `python l08_authz_demo.py` | Yes |
| `l08_auth_server.py` / `l08_auth_client.py` | Bearer-token HTTP server, 401 and 403 behaviour | server, then client | Yes |
| `l09_retry_demo.py`, `l09_timeout_demo.py`, `l09_circuit_breaker.py` | Retry with backoff, timeout, circuit breaker | `python <file>` | Yes |
| `l09_mcp_reliable_call.py` | Timeout and bounded retry around `call_tool` | `python l09_mcp_reliable_call.py` | Yes |
| `l10_claims_server.py`, `l10_discover_tools.py` | MCP tools as LangChain tools | `python l10_discover_tools.py` | Yes |
| `l10_agent_scripted_test.py` | LangGraph agent loop with a scripted model | `python l10_agent_scripted_test.py` | Yes |
| `l10_claims_agent.py` | Agent with Azure OpenAI | needs `.env` credentials | Everything except the LLM call |
| `l11_server.py`, `l11_workflow.py` | Tool, Resource and Prompt in one workflow | `python l11_workflow.py --dry-run` | Yes; the final LLM call needs credentials |
| `adv_progress_server.py` / `adv_progress_client.py` | Progress and log notifications | `python adv_progress_client.py` | Yes |

Tested environment: Windows 11, Python 3.13.13, `mcp` 1.30.0, `langchain-mcp-adapters` 0.3.2, `langgraph` 1.2.17, `langchain-core` 1.7.0, `langchain-openai` 1.7.2.